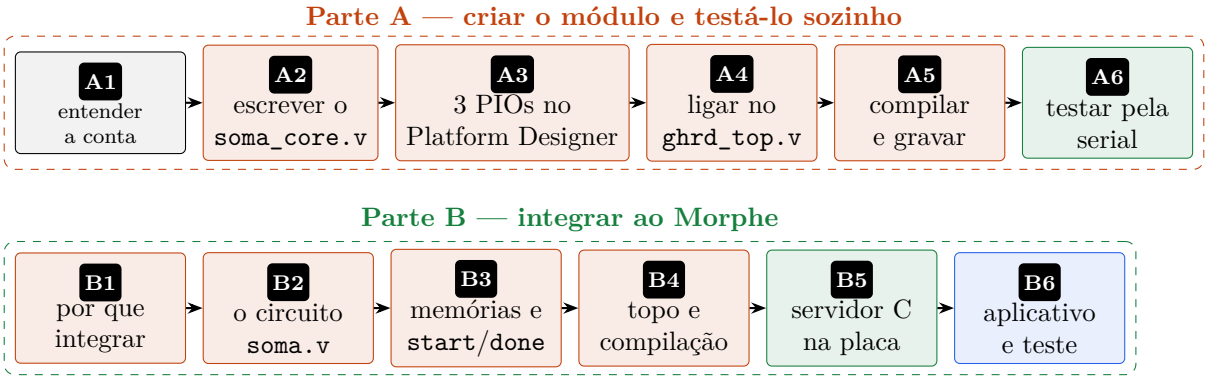
Como criar um módulo no Quartus e ligá-lo ao Morphe

Roteiro prático — exemplo: a soma de dois sinais

Antonio Nicassio Santos Lima Estágio LabPS/UEFS — supervisão: Prof. Armando S. Sanca

outubro de 2026

Este roteiro cria, do zero, um bloco de hardware para a FPGA da DE1-SoC e o põe para trabalhar no Morphe. O exemplo é $y[n] = a[n] + b[n]$: a conta é trivial de propósito, para a atenção ficar no **caminho**, que é o mesmo para qualquer acelerador. Ele tem duas partes:



- **Parte A**: você escreve o módulo no Quartus e o testa **sozinho**, um par de números por vez, digitando os valores pela serial da placa. Nada de servidor nem de aplicativo: se o resultado estiver errado, o erro só pode estar no módulo.
- **Parte B**: o mesmo módulo passa a processar sinais inteiros (1024 amostras) e ganha um botão no aplicativo do Morphe. O código desta parte está pronto no gabarito; o roteiro explica o que cada peça faz e por que existe, para você saber o que mudar no seu próprio módulo.

Antes de começar

O código

Na estação ou num computador do laboratório, num terminal. Digite `bash` primeiro: alguns terminais abrem em `ksh`, e nele os comandos do Quartus não são encontrados.

Faça

Um clone **seu**, na sua pasta pessoal, a partir da branch `estagio/adc-fft` (a versão do Morphe só com o ADC e a FFT). Um comando por vez:

```
cd ~
```

```
git clone -b estagio/adc-fft https://github.com/nicassiosantos/morphe.git meu-modulo
```

```
cd meu-modulo
```

```
git switch -c modulo-soma-teste
```

O que é e por quê

O `/opt/morphe` é a cópia que os alunos usam: mexer nela muda o Morphe de quem estiver na sala. O clone na sua pasta é só seu. A branch nova (`git switch -c`) guarda as suas mudanças separadas da `estagio/adc-fft`.

Já existe: reaproveite

O **gabarito** (a soma pronta) veio junto, na branch `origin/estagio/modulo-soma-adc-fft`. Para ver o que ela muda num arquivo, ou trazer um arquivo pronto:

```
git diff estagio/adc-fft origin/estagio/modulo-soma-adc-fft -- Quartus/ghrd_top.v
```

```
git show origin/estagio/modulo-soma-adc-fft:Quartus/soma_core.v > Quartus/soma_core.v
```

Cuidado

Copiar código deste PDF pode juntar tudo numa linha só (depende do leitor de PDF). Os mesmos trechos estão em texto puro no `docs/roteiro-modulo/trechos.txt` do gabarito, na ordem em que aparecem aqui; copie de lá:

```
git show origin/estagio/modulo-soma-adc-fft:docs/roteiro-modulo/trechos.txt
```

As pastas e os arquivos

```
meu-modulo/
├── Quartus/ ..... FPGA — abrir no Quartus 20.1
│   ├── soc_system.qpf ..... o projeto: é este que se abre
│   ├── soma_core.v ..... NOVO A2 — a conta
│   ├── soma.v ..... NOVO B2 — a conta sobre 1024 amostras
│   ├── soc_system.qsys ..... MUDAR A3 e B3 (pelo Platform Designer)
│   ├── ghrd_top.v ..... MUDAR A4 e B4
│   ├── soc_system.qsf ..... MUDAR A2 e B2 (lista de arquivos)
│   ├── soc_system/, soc_system.sopcinfo ..... GERADO (não editar) A5
│   ├── output_files/soc_system_time_limited.sof ..... GERADO (não editar) A5 (o bitstream)
│   ├── gen_hps_header.py ..... script que gera o hps_0.h
│   └── testbench/tb_soma_core.v ..... simulação do núcleo (opcional, A2)
├── C/ ..... servidor que roda no ARM da placa
│   ├── hps_0.h ..... GERADO (não editar) A5 — endereços de tudo
│   └── morphe_server.c e outros 3 ..... MUDAR B5
├── Python/ ..... aplicativo do aluno
│   ├── soma_window.py ..... NOVO B6
│   └── morphe_protocol.py e outros 2 ..... MUDAR B6
└── morphe-up.sh ..... grava a placa e atualiza o servidor
```

O que está marcado GERADO (não editar) sai das ferramentas: se for editado à mão, a próxima geração apaga — ou pior, a placa e o servidor deixam de concordar.

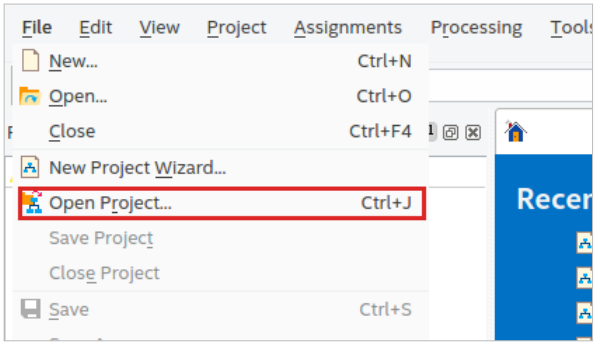
Abrir o projeto no Quartus

Cuidado

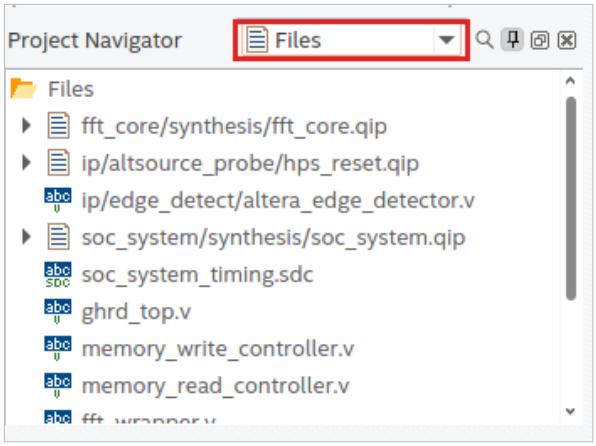
Sempre o Quartus 20.1: no menu de aplicativos, **Quartus Prime Lite 20.1**; ou, no terminal, `quartus &`. Os prints deste roteiro são do Quartus 20.1 de uma estação do laboratório: é o que você vai ver. Se aparecer a faixa amarela *IP upgrade recommended* ou avisos *Used . . . 23.1 (instead of 20.1)*, você abriu a versão errada: feche **sem salvar**.

Faça

File → **Open Project. . .** → `~/meu-modulo/Quartus/soc_system.qpf`. Depois, no alto do *Project Navigator*, troque **Hierarchy** por **Files**: aparece a lista de arquivos do projeto.



File → **Open Project. . .**



Files: os arquivos do projeto

Parte A — criar o módulo e testá-lo sozinho

A1 Entender a conta

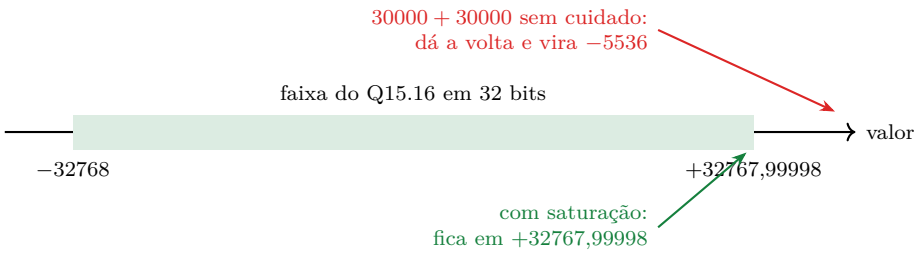
O que é e por quê

A FPGA não tem ponto flutuante: ela soma **inteiros**. O Morphe guarda cada número real como um inteiro de 32 bits em **Q15.16**: 16 bits depois da vírgula. O inteiro guardado é o valor vezes $2^{16} = 65536$:

$$1{,}5 \;\rightarrow\; 1{,}5 \times 65536 = 98304 = \texttt{0x00018000}$$

Negativos usam complemento de dois, como todo inteiro com sinal. A menor fração que cabe é $1/65536 \approx 0{,}000015$, e a faixa vai de -32768 a $+32767{,}99998$.

Somar dois Q15.16 dá outro Q15.16: a vírgula não sai do lugar, então $y = a + b$ é uma soma de inteiros comum. O único cuidado é o **estouro**:



Por isso o módulo faz a conta em 33 bits (ali não estoura) e depois **satura**: se passou do máximo,

devolve o máximo; se passou do mínimo, o mínimo. Num sinal de áudio, saturar soa como um corte; dar a volta soaria como um estalo.

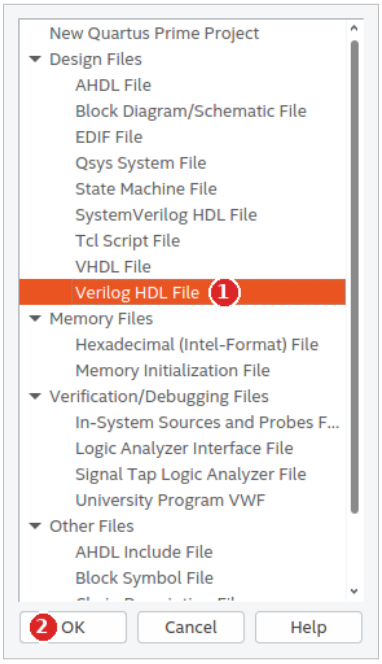
A2 Escrever o `soma_core.v`

O que é e por quê

O `soma_core.v` é **o módulo de verdade**: só a conta, um par de números por vez, sem memória nem relógio. Tudo o que vem depois (PIOs, memórias, servidor) só leva números até ele e traz o resultado de volta. Mantê-lo separado é o que permite testá-lo sozinho na A6.

Faça

1. **File** → **New...** → (1) **Verilog HDL File** → (2) **OK**. Abre um editor vazio.
2. Digite o módulo abaixo.
3. **File** → **Save As...**: nome `soma_core.v`, na pasta `Quartus/` (a mesma do `soc_system.qpf`). Deixe marcada a opção *Add file to current project*: é ela que põe o arquivo no projeto.



File → New...

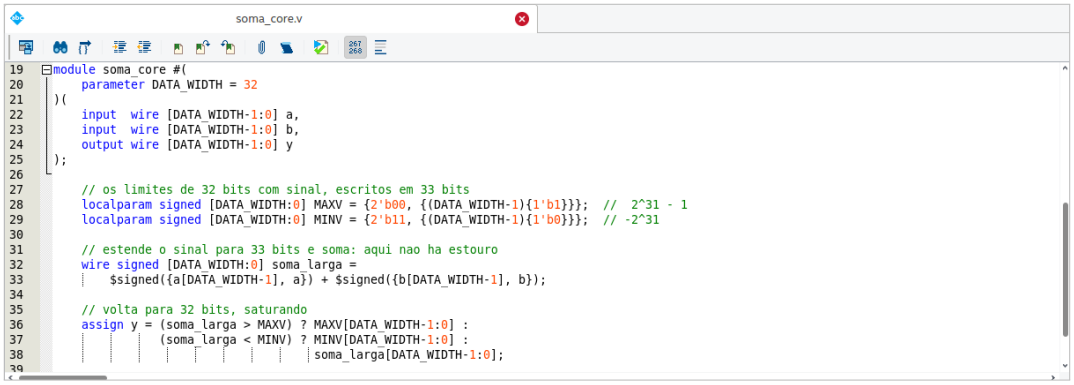
```
module soma_core #(
    parameter DATA_WIDTH = 32
)(
    input  wire [DATA_WIDTH-1:0] a,     // Q15.16
    input  wire [DATA_WIDTH-1:0] b,     // Q15.16
    output wire [DATA_WIDTH-1:0] y      // Q15.16, saturado
);
    // os limites de 32 bits com sinal, escritos em 33 bits
    localparam signed [DATA_WIDTH:0] MAXV = {2'b00, {(DATA_WIDTH-1){1'b1}}};  //  2^31 - 1
    localparam signed [DATA_WIDTH:0] MINV = {2'b11, {(DATA_WIDTH-1){1'b0}}};  // -2^31

    // estende o sinal para 33 bits e soma: aqui nao ha estouro
    wire signed [DATA_WIDTH:0] soma_larga =
        $signed({a[DATA_WIDTH-1], a}) + $signed({b[DATA_WIDTH-1], b});

    // volta para 32 bits, saturando
    assign y = (soma_larga > MAXV) ? MAXV[DATA_WIDTH-1:0] :
```

```
                (soma_larga < MINV) ? MINV[DATA_WIDTH-1:0] :
                                      soma_larga[DATA_WIDTH-1:0];
endmodule
```

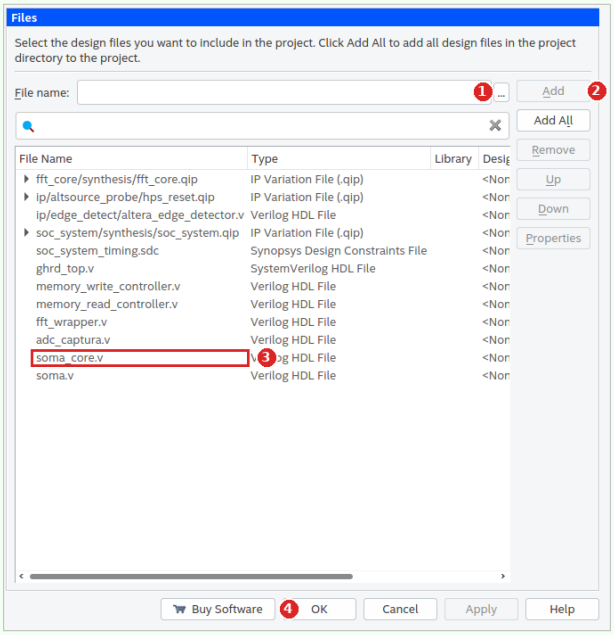
- `{a[31], a}` repete o bit de sinal à esquerda: é assim que um número com sinal ganha um bit sem mudar de valor.
- `assign` sem relógio: y muda junto com a e b. Um módulo assim se chama *combinacional*.



o `soma_core.v` no editor do Quartus (o do gabarito tem um cabeçalho de comentários)

Confira

Project → **Add/Remove Files in Project...**: o `soma_core.v` está na lista 3. Se não estiver, 1 `...` para escolhê-lo, 2 **Add** e 4 **OK**.



os arquivos do projeto (esta é a lista do gabarito, que já tem também o `soma.v` da Parte B)

Já existe: reaproveite

Opcional: simular antes de compilar. Em um segundo você vê os casos da A6 passarem, contra 10 minutos de compilação. De dentro de `~/meu-modulo`, traga do gabarito o testbench e o script que o roda:

```
git show origin/estagio/modulo-soma-adc-fft:Quartus/testbench/tb_soma_core.v > Quartus/
    testbench/tb_soma_core.v
```

```
git show origin/estagio/modulo-soma-adc-fft:Quartus/testbench/roda_tb_soma.sh > Quartus/
    testbench/roda_tb_soma.sh
```

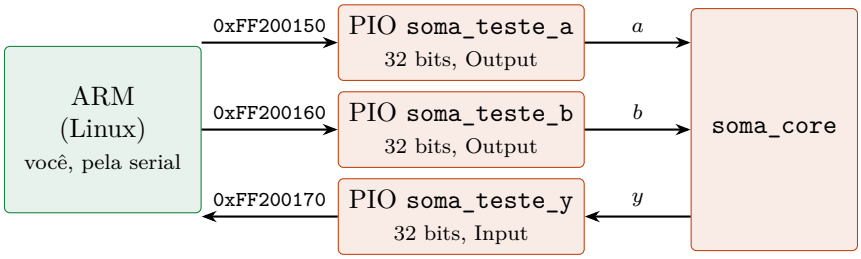
```
bash Quartus/testbench/roda_tb_soma.sh
```

A última linha deve ser `TUDO OK`. O script usa o Icarus Verilog se o computador tiver; senão, o ModelSim que vem com o Quartus 20.1 (é o caso das estações do laboratório).

A3 Três PIOs para conversar com o módulo

O que é e por quê

A placa tem dois mundos: o **HPS** (o processador ARM, com Linux, onde você vai digitar pela serial) e a **FPGA** (onde fica o seu `soma_core`). Eles se falam por **pontes**: endereços de memória do ARM que, na verdade, são registradores dentro da FPGA.
Um **PIO** (*Parallel I/O*) é o registrador mais simples que se põe numa ponte: o ARM escreve num endereço e o valor aparece em fios da FPGA (*Output*), ou a FPGA põe um valor em fios e o ARM lê num endereço (*Input*). Para testar a soma bastam três: a e b saindo do ARM, e y voltando.

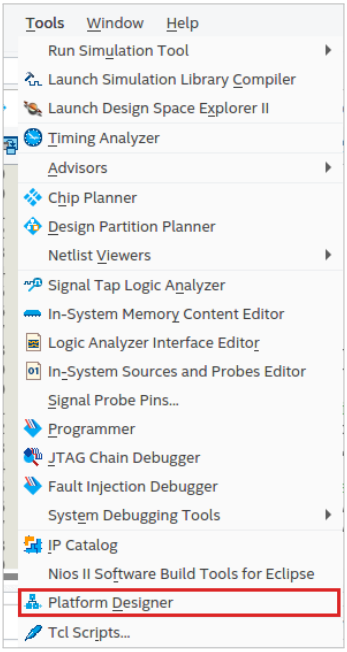


ponte leve (`h2f_lw_axi_master`), que começa em `0xFF200000`

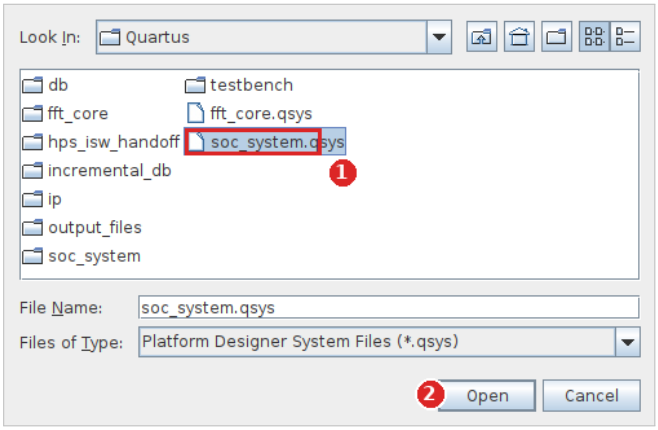
Abrir o sistema

Faça

Com o projeto aberto: **Tools** → **Platform Designer**, e escolha `soc_system.qsys` (não o `fft_core.qsys`, que é a FFT).



Tools → Platform Designer



escolha o `soc_system.qsys`

O que é e por quê

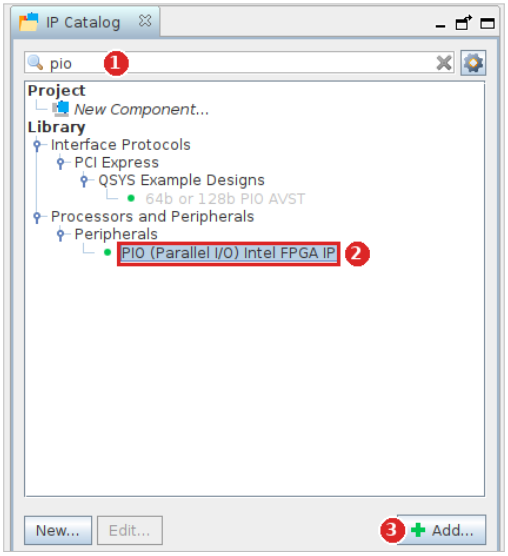
O Platform Designer desenha o **sistema** da placa: o HPS, as pontes e todos os componentes pendurados nelas (as memórias da FFT e do ADC, os PIOs de cada bloco...). Ele gera o Verilog que liga tudo isso e o arquivo `.sopcinfo`, de onde sai o endereço de cada componente.

Acrescentar os três PIOs

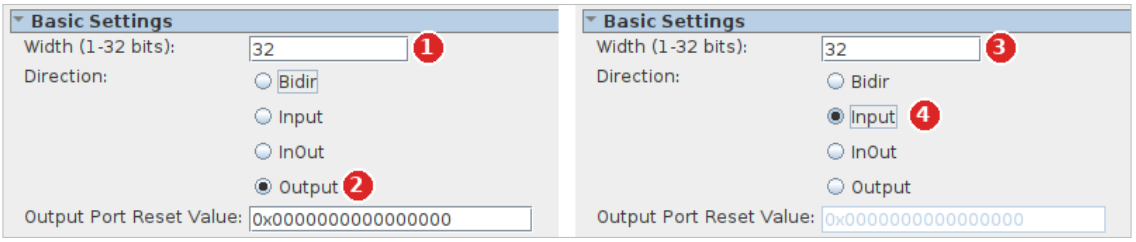
Faça

Para cada PIO:

1. no **IP Catalog**, à esquerda: (1) digite `pio`, (2) escolha *PIO (Parallel I/O) Intel FPGA IP* e (3) clique em **+ Add...**;
2. abre a janela de parâmetros com o nome provisório `pio_0` no topo (é normal: o nome certo vem no item seguinte). Nela: *Width* **32** e a direção — *Output* para `soma_teste_a` e `soma_teste_b`, *Input* para `soma_teste_y` —, e **Finish**;
3. o PIO entra no fim da lista com um nome como `pio_0`: botão direito nele → **Rename** e dê o nome certo.



IP Catalog



1 2 `soma_teste_a` (e `_b`): 32 bits, *Output*; 3 4 `soma_teste_y`: 32 bits, *Input*

Ligar, exportar e dar endereço

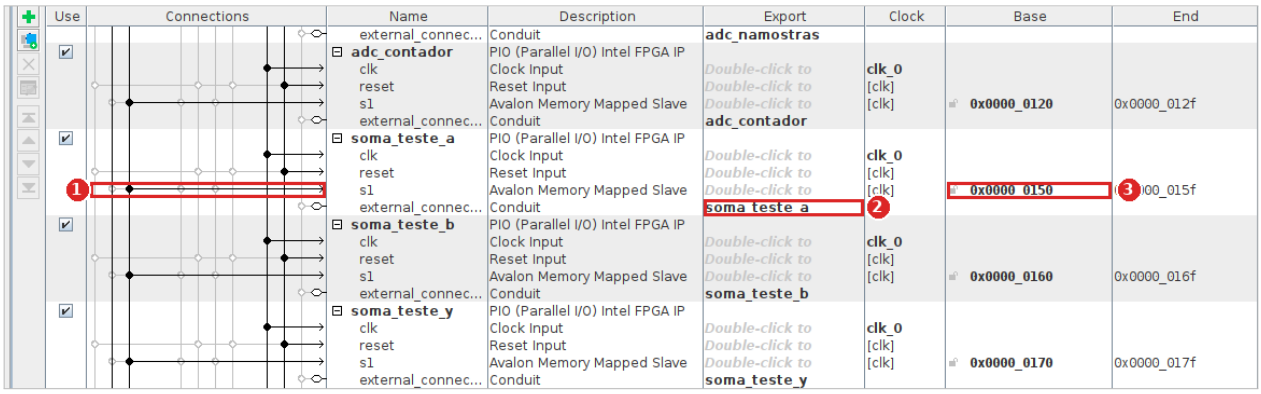
Cada PIO tem duas pontas, e as duas precisam ser ligadas:

Ponta	O que fazer	Por quê
`clk`, `reset`	`clk_0` na coluna **Clock**; `reset` em `clk_0.clk_reset`	todo registrador precisa de relógio e de um estado inicial
`s1` 1	ligar ao `hps_0.h2f_lw_axi_master`	é o lado do ARM: sem isso, o PIO não tem endereço e o ARM não o enxerga
`external_connection` 2	exportar	é o lado dos fios: exportar faz os fios saírem do sistema, para você ligá-los ao `soma_core` na A4
Base 3	`0x150`, `0x160`, `0x170`, e travar o cadeado	é o endereço do PIO dentro da ponte; travado, ele não muda sozinho

Faça

Tudo na aba **System Contents**:

1. 1 na coluna **Connections**, as linhas verticais são os mestres (passe o mouse sobre uma para ver o nome). Clique na bolinha onde a linha `s1` do PIO cruza o `h2f_lw_axi_master`: bolinha **cheia** = ligado;
2. 2 na coluna **Export**, duplo clique em *Double-click to export* na linha `external_connection`; deixe o nome proposto;
3. 3 na coluna **Base**, duplo clique, digite o endereço, Enter, e clique no cadeado;
4. **File** → **Save** (Ctrl+S).



os três PIOs prontos: 1 ligado ao HPS, 2 exportado, 3 endereço travado

Confira

A barra de baixo diz *0 Errors*, e na aba **Address Map** os três aparecem na coluna `h2f_lw_axi_master`, nos endereços escolhidos:

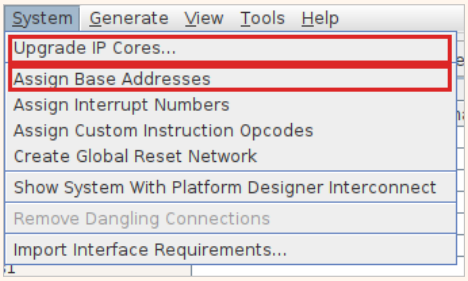
System: soc_system **Path:** pio_0

	fpga_only_master.master	hps_0.h2f_axi_master	hps_0.h2f_lw_axi_master
fft_bfp_exponent.s1			0x0000_0050 - 0x0000_005f
fft_inverse.s1			0x0000_0070 - 0x0000_007f
fft_wrapper_done.s1			0x0000_0060 - 0x0000_006f
fft_wrapper_start.s1			0x0000_0080 - 0x0000_008f
fft_xn_imag.s1			
fft_xn_imag.s2		0x0001_b000 - 0x0001_bfff	
fft_xn_re.s1			
fft_xn_re.s2		0x0001_a000 - 0x0001_afff	
fft_yn_imag.s1			
fft_yn_imag.s2		0x0001_9000 - 0x0001_9fff	
fft_yn_re.s1			
fft_yn_re.s2		0x0001_8000 - 0x0001_8fff	
hps_0.f2h_axi_slave			
intr_capturer_0.avalon_slave_0	0x0003_0000 - 0x0003_0007		
jtag_uart.avalon_jtag_slave	0x0002_0000 - 0x0002_0007		0x0002_0000 - 0x0002_0007
onchip_memory2_0.s1	0x0000_0000 - 0x0000_ffff	0x0000_0000 - 0x0000_ffff	
sysid_qsys.control_slave	0x0001_0000 - 0x0001_0007		0x0001_0000 - 0x0001_0007
adc_buf.s1			
adc_buf.s2		0x0002_0000 - 0x0003_ffff	
adc_start.s1			0x0000_00d0 - 0x0000_00df
adc_done.s1			0x0000_00e0 - 0x0000_00ef
adc_config.s1			0x0000_00f0 - 0x0000_00ff
adc_divisor.s1			0x0000_0100 - 0x0000_010f
adc_namostras.s1			0x0000_0110 - 0x0000_011f
adc_contador.s1			0x0000_0120 - 0x0000_012f
soma_teste_a.s1			0x0000_0150 - 0x0000_015f
soma_teste_b.s1			0x0000_0160 - 0x0000_016f
soma_teste_y.s1			0x0000_0170 - 0x0000_017f

Address Map

Cuidado

Nunca use System → Assign Base Addresses nem **Upgrade IP Cores**. O primeiro muda o endereço de *todos* os componentes — e o servidor da placa, que conhece os endereços antigos, passa a escrever no lugar errado. O segundo troca a versão dos IPs. Os endereços novos se escolhem à mão, nos buracos livres (veja o **Address Map**).

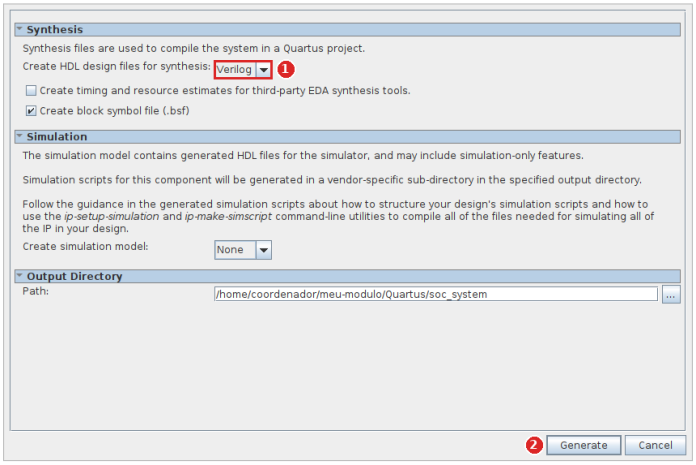


os dois itens que **não** se usam

Gerar

Faça

Generate → **Generate HDL...** (ou o botão **Generate HDL...** no canto de baixo) → ❶ *Verilog*, pasta de saída como está, ❷ **Generate**. Espere *Generate Completed* e feche com **Finish**.



Generate HDL

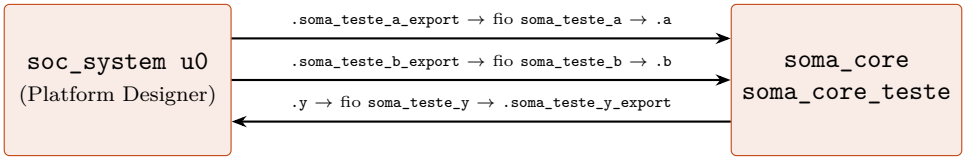
O que é e por quê

Gerar escreve o Verilog do sistema (`soc_system/synthesis/soc_system.v`) e o `soc_system.sopcinfo`. É nesse Verilog que os PIOs exportados viram **portas**: `soma_teste_a_export`, `soma_teste_b_export` e `soma_teste_y_export`.

A4 Ligar o módulo no topo: `ghrd_top.v`

O que é e por quê

O `ghrd_top.v` é o arquivo de cima do projeto: é ele que vira o chip. Lá dentro há uma instância do sistema do Platform Designer, chamada `u0`. Os PIOs exportados são portas de `u0`; o seu `soma_core` é outra instância. Faltam só os **fios** entre eles.



Faça

Abra o `ghrd_top.v` (duplo clique nele em **Files**). Três acréscimos — use Ctrl+F para achar os lugares:

1. **os fios**: junto dos outros `wire`, por exemplo depois da linha que tem `adc_contador;`

```
// --- teste isolado do nucleo (roteiro, Parte A): tres PIOs de 32 bits ---
wire [31:0] soma_teste_a;    // HPS -> FPGA
wire [31:0] soma_teste_b;    // HPS -> FPGA
wire [31:0] soma_teste_y;    // FPGA -> HPS
```

2. **as portas de `u0`**: dentro de `soc_system u0 (`, junto dos outros `_export` (procure `.adc_contador_export`). Atenção à vírgula no fim de cada linha:

```
    .soma_teste_a_export     (soma_teste_a),
```

```
    .soma_teste_b_export     (soma_teste_b),
    .soma_teste_y_export     (soma_teste_y),
```

3. **o módulo**: antes do `endmodule` do fim do arquivo:

```
soma_core #(
    .DATA_WIDTH (32)
) soma_core_teste (
    .a (soma_teste_a),
    .b (soma_teste_b),
    .y (soma_teste_y)
);
```

Salve (Ctrl+S).

O que é e por quê

`.soma_teste_a_export (soma_teste_a)` quer dizer "a porta `soma_teste_a_export` de `u0` fica ligada ao fio `soma_teste_a`". O mesmo fio aparece em `.a (soma_teste_a)` do seu módulo: é assim que os dois ficam conectados. O nome da porta tem de ser **exatamente** o que o Platform Designer gerou; confira com `grep -n soma_teste soc_system/synthesis/soc_system.v`.

A5 Compilar, gerar o `hps_0.h` e gravar a placa

Faça

1. No Quartus: **Processing** → **Start Compilation** (Ctrl+L). Leva uns 10 minutos.
2. No terminal, de dentro de `~/meu-modulo/Quartus/`:

```
python3 gen_hps_header.py
```

3. Traga do gabarito o `devmem` da A6 (o programa, a regra `make devmem` e o `morphe-up.sh` que o envia à placa), de dentro de `~/meu-modulo`:

```
git checkout origin/estagio/modulo-soma-adc-fft -- C/devmem.c C/Makefile morphe-up.sh
```

4. Na **estação** (é onde estão os cabos das placas), de dentro de `~/meu-modulo`, fora do horário de aula, grave a placa. No lugar de `CABO`, o nome do cabo da **sua** placa, entre aspas simples; no lugar de `PLACA`, o nome dela (os dois quadros abaixo):

```
./morphe-up.sh --cable 'CABO' --board PLACA --deploy
```

Faça

Qual é o cabo da sua placa. Cada placa tem o seu USB-Blaster, e o Quartus dá a cada um o nome `DE-SoC [1-N]`. O número entre colchetes é a porta USB da estação: **muda** se o cabo trocar de porta. Por isso não há um nome fixo para copiar. Para descobrir:

1. liste os cabos que a estação enxerga:

```
jtagconfig
```

   Cada cabo aparece numa linha como `1) DE-SoC [1-2]`; o nome é o que vem depois do `1)`, com os colchetes.

2. se houver mais de um, **desconecte da estação** o USB-Blaster da sua placa (a USB que *não* é a marcada UART), rode `jtagconfig` de novo e veja qual nome sumiu: é o seu. Ligue o cabo de volta e confira que o nome voltou igual.

Se o cabo não for o da placa do `--board`, o `morphe-up.sh` recusa e diz qual placa encontrou — o servidor de outra placa não é mexido.

Faça

Qual é a serial e o nome da sua placa. A placa tem duas USB: a do USB-Blaster (o cabo, acima) e a marcada **UART**, que é a **serial** do ARM. Pela serial você vê o nome da placa, e na A6 vai usá-la para o teste.

1. com o cabo UART **desligado** do computador, rode o comando abaixo; ligue o cabo e rode de novo. Os nomes que apareceram (`/dev/ttyUSB0`, `/dev/ttyUSB1`...) são os da sua placa. Algumas placas criam **dois** de uma vez: a serial é o de número menor.

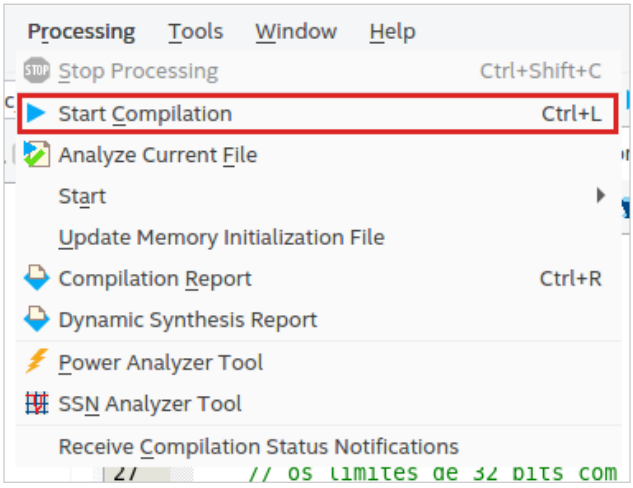
   ```
   ls /dev/ttyUSB*
   ```

2. abra a serial, trocando `/dev/ttyUSB0` pelo nome que você achou:

   ```
   screen /dev/ttyUSB0 115200
   ```

3. dê Enter. Aparece uma linha como `de1soc-02 login:`: o nome da placa é o que vem antes de `login:` (aqui, `de1soc-02`). É esse o `PLACA` do comando acima.
4. para sair do `screen`: Ctrl+A, depois K, depois y.

Sem o `--board`, o `morphe-up.sh` grava a FPGA e para antes de mandar o servidor, com *não sei qual placa é a deste comando*: ele acha uma placa na rede, mas não tem como saber se é a do seu cabo. Se ele disser que a placa *está em uso*, alguém a usou no último minuto: espere, ou confira com quem está na sala.



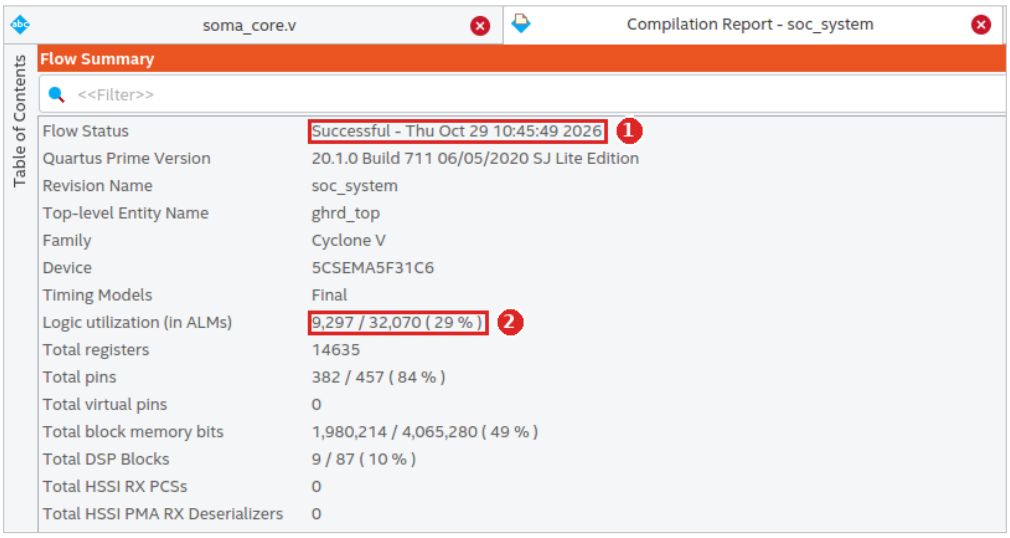
Processing → Start Compilation

O que é e por quê

- **Compilar** transforma o projeto no arquivo `output_files/soc_system.sof`, a configuração da FPGA (o *bitstream*).
- O `gen_hps_header.py` lê o `.sopcinfo` e escreve o `C/hps_0.h`: a lista de endereços de todos os componentes. O servidor da placa é compilado com ele; por isso o servidor e o *bitstream* sempre andam juntos.
- O `morphe-up.sh` grava o `.sof` na FPGA pelo cabo, mantém a licença do Quartus ativa (sem ela, a placa para depois de uma hora) e, com `-deploy`, recompila o servidor da placa com o `hps_0.h` novo.

Confira

- o Quartus diz *Full Compilation was successful*;
- `grep SOMA_TESTE ../C/hps_0.h` mostra `0x150`, `0x160` e `0x170`;
- o `morphe-up.sh` termina com os testes `4/4`.



Processing → **Compilation Report** (Ctrl+R), **Flow Summary**: 1 *Successful*; 2 a lógica usada: 25 % só com a Parte A, 29 % com a Parte B (este print).

Confira

Assim termina o `morphe-up.sh` na placa 2 (29/10/2026). Os avisos *setlocale* e *Clock skew* são normais: a placa não tem relógio e acorda em 1970.

```
    ok   servidor compilado na placa
==> (re)iniciando o servidor
    ok   servidor no ar na porta 5000
    ok   a placa confirmou o bitstream (assinatura MRPH): o cabo DE-SoC [1-1] esta nela
==> verificando a plataforma (morphe_ping)
[1] TCP reachable
[2] Protocolo Morphe respondendo
[3] Captura do ADC (64 amostras do CH0 a 20 kHz)
[4] FFT com impulso (x = δ[n] → |X[k]| constante)
Todos os 4 testes passaram. Morphe está operacional.
    ok   os quatro testes passaram
```

A6 Testar pela serial

O que é e por quê

A placa tem duas USB: a do USB-Blaster, que grava a FPGA, e a marcada **UART**, que é a **serial** do ARM. Por ela você entra no Linux da placa, sem rede. Lá dentro, o `devmem` lê e escreve um endereço físico — exatamente o que o ARM faz para falar com os PIOs. É o teste mais direto possível: você escreve a e b, lê y, e não há nenhum programa do Morphe no meio.
O Linux das placas não traz o `devmem` de costume; o Morphe tem o seu (`C/devmem.c`, mesma sintaxe), que o `--deploy` da A5 compilou na pasta `~/morphe` da placa.

Faça

Abra a serial que você achou na A5, trocando `/dev/ttyUSB0` pelo nome dela:

```
screen /dev/ttyUSB0 115200
```

Dê Enter. No `login:`, digite `root`; no `Password:`, a senha `morphe` (ela não aparece enquanto você digita). Para sair do `screen`: Ctrl+A, depois K, depois y.

Faça

Dentro da placa, entre na pasta do Morphe:

```
cd ~/morphe
```

Para cada linha da tabela: escreva a, escreva b, leia y. O primeiro caso, $1{,}5 + 2{,}25$:

```
./devmem 0xFF200150 32 0x00018000
```

```
./devmem 0xFF200160 32 0x00024000
```

```
./devmem 0xFF200170
```

A placa responde `0x0003C000`, que é 3,75.

a	b	a em hex	b em hex	y esperado	y
1,5	2,25	`0x00018000`	`0x00024000`	`0x0003C000`	3,75
−1	0,5	`0xFFFF0000`	`0x00008000`	`0xFFFF8000`	−0,5
100	−100	`0x00640000`	`0xFF9C0000`	`0x00000000`	0
30000	30000	`0x75300000`	`0x75300000`	`0x7FFFFFFF`	32767,99998 (saturou)
−30000	−30000	`0x8AD00000`	`0x8AD00000`	`0x80000000`	−32768 (saturou)

Já existe: reaproveite

Para converter outros valores, no computador (não na placa):

```
python3 -c "print(hex(round(1.5 * 65536) & 0xFFFFFFFF))"
```

```
python3 -c "v = 0xFFFF8000; print((v - 2**32 if v >> 31 else v) / 65536)"
```

Confira

Os cinco casos batem. Os dois últimos são os que importam: um estouro que **não** satura daria um número de sinal trocado. Se algum caso falhar, o erro está no `soma_core.v` ou nas ligações da A3/A4 — não há mais nada no caminho.

Cuidado

- Só use o `devmem` nesses endereços **depois** de gravar a placa com o seu `.sof` (A5). Ler ou escrever um endereço da ponte onde não há componente **trava a placa**: ela some da rede até ser desligada e ligada de novo.
- Se a serial não mostrar nada nem com Enter, essa placa está sem login na serial; entre pela rede (`ssh root@`*IP da placa*) e use os mesmos comandos.
- Se `./devmem` não existir, compile-o na placa: `cd ~/morphe && make devmem`.

Confira

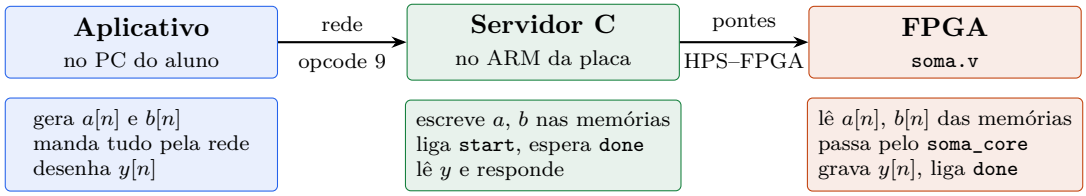
Assim ficou o teste na placa 2 (29/10/2026):

```
root@de1soc-02:~/morphe# ./devmem 0xFF200150 32 0x00018000
root@de1soc-02:~/morphe# ./devmem 0xFF200160 32 0x00024000
root@de1soc-02:~/morphe# ./devmem 0xFF200170
0x0003C000
root@de1soc-02:~/morphe# ./devmem 0xFF200150 32 0xFFFF0000; ./devmem 0xFF200160 32 0x00008000;
    ./devmem 0xFF200170
0xFFFF8000
root@de1soc-02:~/morphe# ./devmem 0xFF200150 32 0x00640000; ./devmem 0xFF200160 32 0xFF9C0000;
    ./devmem 0xFF200170
0x00000000
root@de1soc-02:~/morphe# ./devmem 0xFF200150 32 0x75300000; ./devmem 0xFF200160 32 0x75300000;
    ./devmem 0xFF200170
0x7FFFFFFF
root@de1soc-02:~/morphe# ./devmem 0xFF200150 32 0x8AD00000; ./devmem 0xFF200160 32 0x8AD00000;
    ./devmem 0xFF200170
0x80000000
```

Parte B — integrar ao Morphe

B1 Por que integrar, e o que muda

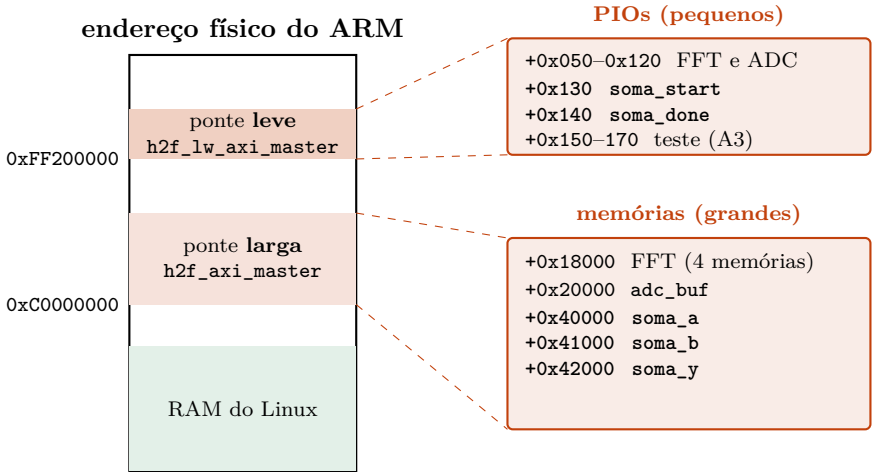
O `soma_core` funciona. Mas, do jeito da Parte A, cada amostra custa três comandos digitados — um sinal de 1024 amostras seriam mais de 3000. Integrar ao Morphe é fazer o caminho que os blocos dele (a FFT e o ADC) já fazem:



O que entra	Por quê
3 **memórias** (a, b, y), 1024 palavras cada	o ARM escreve o sinal inteiro de uma vez, pela ponte larga, em vez de 1024 PIOs
PIOs `start` e `done`	o ARM avisa "pode começar" e o circuito avisa "terminei": sem isso, o ARM leria y antes de ficar pronto
`soma.v`	a máquina de estados que percorre as 1024 amostras, usando o `soma_core` da Parte A
um **opcode** no servidor	o servidor recebe pedidos pela rede; cada operação tem um número (a soma é o 9)
uma **janela** no aplicativo	para o aluno gerar sinais e ver o resultado sem digitar nada

Os números que precisam concordar em todas as camadas:

O quê	Valor	Onde aparece
opcode	9	`morphe_protocol.h` e `morphe_protocol.py`
amostras	1024	`morphe_config.h`, `morphe_config.py`, `N_SAMPLES` do Verilog
memórias a, b, y	4 KiB cada	`0x40000`, `0x41000`, `0x42000` (ponte larga)
PIOs `start`, `done`	1 bit cada	`0x130`, `0x140` (ponte leve)



O que é e por quê

Duas pontes porque são dois tipos de tráfego: a **larga** (128 bits, mais rápida) para mover blocos de dados, a **leve** (32 bits) para registradores de controle. O endereço do Platform Designer é sempre um **deslocamento** dentro da ponte: `soma_a` está em `0xC0000000` + `0x40000`.

B2 O circuito: `soma.v`

O que é e por quê

O `soma.v` não faz conta nenhuma: ele **envolve** o `soma_core`. Espera o `start`, e para $n = 0, 1, \ldots, 1023$ lê $a[n]$ e $b[n]$ das memórias, passa pelo núcleo e grava $y[n]$. No fim, liga o `done`. Esse "envelope" é o mesmo para qualquer módulo amostra a amostra: troque o `soma_core` por outra conta e o resto fica.

Faça

Traga do gabarito e ponha no projeto (como na A2, **Project → Add/Remove Files**):

```
git show origin/estagio/modulo-soma-adc-fft:Quartus/soma.v > Quartus/soma.v
```

Faça

Ou, se preferir, crie o `Quartus/soma.v` no editor do Quartus (como na A2) e cole o código completo abaixo. Se o PDF juntar as linhas ao copiar, use o `trechos.txt` (página 2).

```
// ============================================================================
// ============================================================================
// soma.v -- soma de dois sinais, amostra a amostra: y[n] = a[n] + b[n].
//
// Modulo de exemplo do roteiro "Como adicionar um modulo ao Morphe"
// (docs/roteiro-modulo). A conta e trivial de proposito: o que interessa e o
// caminho inteiro -- memorias, PIOs, servidor e cliente --, que e o mesmo de
// qualquer acelerador.
//
// Como conversa com o resto:
//   - tres memorias de porta dupla do Platform Designer (soma_a, soma_b,
//     soma_y), 1024 palavras de 32 bits cada. O HPS escreve a e b e le y pela
//     porta s2; este modulo usa a porta s1, exportada para o ghrd_top;
//   - dois PIOs: soma_start (HPS -> FPGA) e soma_done (FPGA -> HPS).
//
// Protocolo de start/done, o mesmo da FFT (fft_wrapper.v):
//   1. o HPS escreve as entradas e poe start = 1 (borda de subida dispara);
//   2. o modulo processa SEMPRE N_SAMPLES amostras (o servidor completa com
//      zeros o que o cliente nao mandou) e entao poe done = 1;
//   3. o HPS le a saida e poe start = 0; com start em 0, done volta a 0.
//
// A conta (Q15.16, com saturacao) fica no soma_core.v; este modulo so le a[n]
// e b[n] das memorias, passa pelo nucleo e grava y[n].
//
// Leitura e escrita das memorias: os mesmos memory_read_controller e
// memory_write_controller que o fft_wrapper.v usa. Cada leitura
// custa 6 ciclos e cada escrita 4, contando a ida e a volta pela maquina de
// estados; a e b sao lidas em paralelo, por dois controladores. Uma amostra
// custa 12 ciclos: 1024 amostras em 12 289 ciclos, ~0,25 ms a 50 MHz.
// ============================================================================

`timescale 1 ns / 1 ps

module soma #(
    parameter DATA_WIDTH  = 32,
    parameter N_SAMPLES   = 1024,
    parameter ADDR_N_BITS = 10
)(
    input  wire                     clk,
    input  wire                     reset_n,
    input  wire                     start,
    output reg                      done,

    // --- memoria da entrada a[n] (so leitura) ---
    input  wire [DATA_WIDTH-1:0]    a_sram_readdata,
    output wire [ADDR_N_BITS-1:0]   a_sram_address,
    output wire                     a_sram_chipselect,
    output wire                     a_sram_clken,
    output wire                     a_sram_write,

    // --- memoria da entrada b[n] (so leitura) ---
```

```
    input  wire [DATA_WIDTH-1:0]    b_sram_readdata,
    output wire [ADDR_N_BITS-1:0]   b_sram_address,
    output wire                     b_sram_chipselect,
    output wire                     b_sram_clken,
    output wire                     b_sram_write,

    // --- memoria da saida y[n] (so escrita) ---
    output wire [ADDR_N_BITS-1:0]   y_sram_address,
    output wire [DATA_WIDTH-1:0]    y_sram_wdata,
    output wire                     y_sram_chipselect,
    output wire                     y_sram_clken,
    output wire                     y_sram_write
);

    localparam [2:0] S_IDLE       = 3'd0;
    localparam [2:0] S_READ       = 3'd1;  // pede a[n] e b[n]
    localparam [2:0] S_WAIT_READ  = 3'd2;
    localparam [2:0] S_WRITE      = 3'd3;  // grava y[n]
    localparam [2:0] S_WAIT_WRITE = 3'd4;
    localparam [2:0] S_DONE       = 3'd5;

    reg [2:0] state;
    reg [ADDR_N_BITS-1:0] n;    // amostra corrente

    // start dispara na borda de subida: um start que ficou em 1 da rodada
    // anterior nao dispara outra.
    reg start_d;
    wire start_pulse = start & ~start_d;
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) start_d <= 1'b0;
        else          start_d <= start;
    end

    // --- controle das memorias ---
    reg                     rd_start;
    wire [DATA_WIDTH-1:0]   a_data, b_data;
    wire                    a_done, b_done;

    reg                     wr_start;
    reg  [DATA_WIDTH-1:0]   wr_data;
    wire                    wr_done;

    // --- a conta: o mesmo nucleo testado sozinho na Parte A do roteiro ---
    wire [DATA_WIDTH-1:0] soma_sat;

    soma_core #(.DATA_WIDTH(DATA_WIDTH)) u_conta (
        .a (a_data),
        .b (b_data),
        .y (soma_sat)
    );

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            state    <= S_IDLE;
            done     <= 1'b0;
            n        <= 0;
            rd_start <= 1'b0;
            wr_start <= 1'b0;
            wr_data  <= 0;
        end else begin
            rd_start <= 1'b0;
            wr_start <= 1'b0;

            case (state)
                S_IDLE: begin
                    done <= 1'b0;
                    if (start_pulse) begin
                        n     <= 0;
                        state <= S_READ;
                    end
                end

                // Os dois controladores recebem o mesmo endereco e o mesmo
                // pulso, e terminam no mesmo ciclo.
                S_READ: begin
```

```
                rd_start <= 1'b1;
                state    <= S_WAIT_READ;
            end

            S_WAIT_READ: begin
                if (a_done && b_done) begin
                    wr_data <= soma_sat;
                    state   <= S_WRITE;
                end
            end

            S_WRITE: begin
                wr_start <= 1'b1;
                state    <= S_WAIT_WRITE;
            end

            S_WAIT_WRITE: begin
                if (wr_done) begin
                    if (n == N_SAMPLES - 1) begin
                        state <= S_DONE;
                    end else begin
                        n     <= n + 1'b1;
                        state <= S_READ;
                    end
                end
            end

            // done fica em 1 ate o HPS baixar o start.
            S_DONE: begin
                done <= 1'b1;
                if (!start) state <= S_IDLE;
            end

            default: state <= S_IDLE;
        endcase
    end
end

// --- controladores de memoria: os mesmos do fft_wrapper.v ---
memory_read_controller #(
    .DATA_WIDTH         (DATA_WIDTH),
    .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_rd_a (
    .clk (clk), .reset_n (reset_n),
    .start_read    (rd_start),
    .sram_readdata (a_sram_readdata),
    .data_addr     (n),
    .mem_write     (a_sram_write),
    .data_out      (a_data),
    .mem_address   (a_sram_address),
    .read_done     (a_done),
    .clken         (a_sram_clken),
    .chipselect    (a_sram_chipselect)
);

memory_read_controller #(
    .DATA_WIDTH         (DATA_WIDTH),
    .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_rd_b (
    .clk (clk), .reset_n (reset_n),
    .start_read    (rd_start),
    .sram_readdata (b_sram_readdata),
    .data_addr     (n),
    .mem_write     (b_sram_write),
    .data_out      (b_data),
    .mem_address   (b_sram_address),
    .read_done     (b_done),
    .clken         (b_sram_clken),
    .chipselect    (b_sram_chipselect)
);

memory_write_controller #(
    .DATA_WIDTH         (DATA_WIDTH),
    .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_wr_y (
```

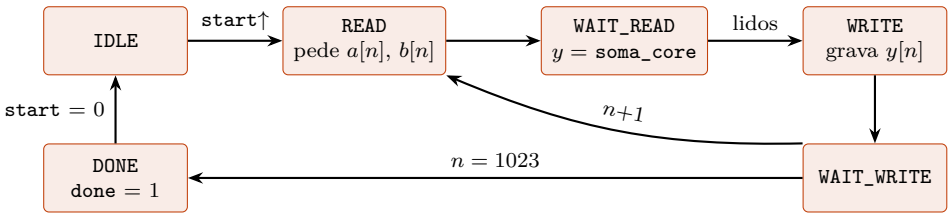
```
        .clk (clk), .reset_n (reset_n),
        .start_write  (wr_start),
        .data_in_addr (n),
        .data_in      (wr_data),
        .write_done   (wr_done),
        .mem_address  (y_sram_address),
        .mem_wdata    (y_sram_wdata),
        .mem_write    (y_sram_write),
        .clken        (y_sram_clken),
        .chipselect   (y_sram_chipselect)
    );

endmodule
```

As portas do `soma.v`, e para que serve cada uma:

Porta	Sentido	Para quê
`clk`, `reset_n`	entra	relógio de 50 MHz e o reset (ativo em 0) do sistema
`start`	entra	vem do PIO `soma_start`: a borda de subida dispara a rodada
`done`	sai	vai ao PIO `soma_done`: fica em 1 até o ARM baixar o `start`
`a_sram_address` [9:0]	sai	qual palavra de a ler: 10 bits porque $2^{10} = 1024$
`a_sram_readdata` [31:0]	entra	a palavra lida, um ciclo depois
`a_sram_chipselect`, `_clken`	sai	"estou usando a memória" — sem eles, ela não responde
`a_sram_write`	sai	sempre 0: a só é lida
`b_sram_...`		o mesmo, para b
`y_sram_address`, `_wdata`	sai	onde gravar e o quê
`y_sram_write`	sai	1 no ciclo em que grava

Por dentro, uma máquina de seis estados:



O núcleo da Parte A entra assim, sem mudar uma linha:

```
wire [DATA_WIDTH-1:0] soma_sat;

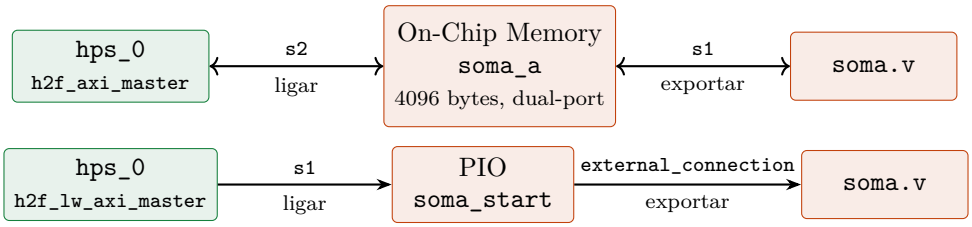
soma_core #(.DATA_WIDTH(DATA_WIDTH)) u_conta (
    .a (a_data),        // a[n], vindo da memoria
    .b (b_data),        // b[n], vindo da memoria
    .y (soma_sat)       // vai para a memoria de y
);
```

Já existe: reaproveite

Ler e escrever as memórias já está feito: `memory_read_controller.v` e `memory_write_controller.v`, os mesmos do `fft_wrapper.v`. Dá-se o endereço e um pulso em `start_read`; o dado volta em `data_out` com um pulso em `read_done`. A soma usa dois de leitura (a e b em paralelo) e um de escrita (y): 1024 amostras em 12 289 ciclos, cerca de 0,25 ms.

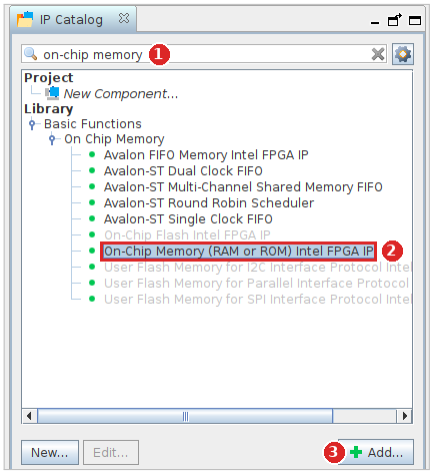
B3 Platform Designer: as memórias e o `start/done`

Cada memória tem **duas portas**: uma para o ARM, outra para o circuito. Os dois usam a mesma memória ao mesmo tempo, sem disputa.

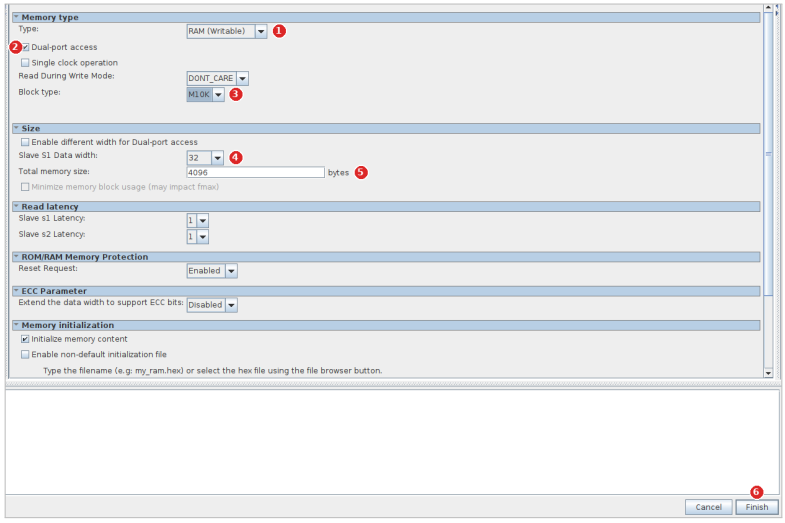


Faça

1. **3 memórias** (`soma_a`, `soma_b`, `soma_y`): no **IP Catalog**, (1) `on-chip memory`, (2) *On-Chip Memory (RAM or ROM) Intel FPGA IP*, (3) **Add**; parâmetros como no print; renomeie.
2. **2 PIOs** de 1 bit: `soma_start` (*Output*) e `soma_done` (*Input*), como na A3.
3. **Ligar**: `s2` das memórias ao `h2f_axi_master`; `s1` dos PIOs ao `h2f_lw_axi_master`; relógios em `clk_0` e resets em `clk_0.clk_reset`. Cada memória *dual-port* tem **dois** de cada (`clk1` e `clk2`, `reset1` e `reset2`, um por porta): ligue os quatro, como nas memórias `fft_xn_re...` da FFT.
4. **Exportar**: `s1` das memórias e `external_connection` dos PIOs.
5. **Endereços** da tabela da B1, travados. Salvar e gerar.



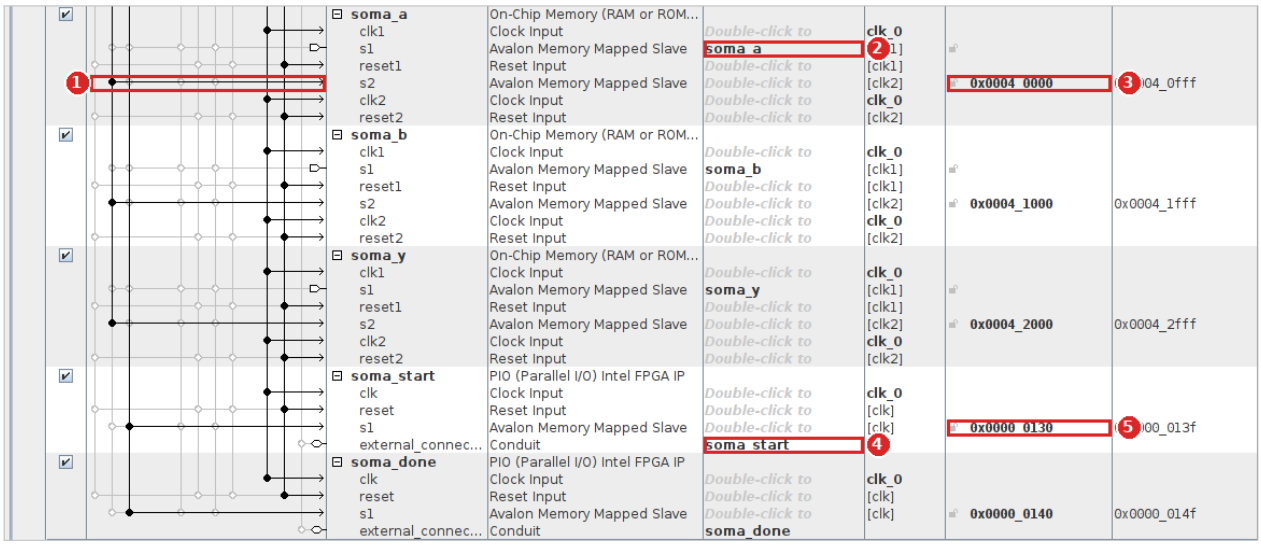
a memória no **IP Catalog**



(1) RAM, (2) *dual-port*, (3) M10K, (4) 32 bits, (5) 4096 bytes, (6) **Finish**

O que é e por quê

- **4096 bytes**: 1024 amostras × 4 bytes (32 bits).
- ***Dual-port***: cria a segunda porta (`s2`), a do ARM. Com uma porta só, o ARM e o circuito disputariam a mesma entrada.
- **M10K**: os blocos de memória de 10 kbits do Cyclone V. A FPGA tem 397 deles; o Morphe usa pouco mais da metade.
- **Ligar o `s2` e exportar o `s1`**: o `s2` fica dentro do sistema, com endereço na ponte larga; o `s1` sai como portas (`soma_a_address`, `soma_a_readdata...`) para o `soma.v`.



os cinco componentes prontos: 1 `s2` ligado ao ARM, 2 `s1` exportado, 3 endereço da memória, 4 PIO exportado, 5 endereço do PIO

Confira

0 Errors na barra de baixo; no **Address Map**, memórias em `h2f_axi_master` e PIOs em `h2f_lw_axi_master`:

System: soc_system **Path:** pio_0

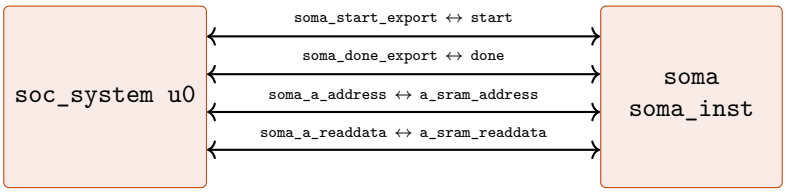
	fpga_only_master.master	hps_0.h2f_axi_master	hps_0.h2f_lw_axi_master
fft_bfp_exponent.s1			0x0000_0050 - 0x0000_005f
fft_inverse.s1			0x0000_0070 - 0x0000_007f
fft_wrapper_done.s1			0x0000_0060 - 0x0000_006f
fft_wrapper_start.s1			0x0000_0080 - 0x0000_008f
fft_xn_imag.s1			
fft_xn_imag.s2		0x0001_b000 - 0x0001_bfff	
fft_xn_re.s1			
fft_xn_re.s2		0x0001_a000 - 0x0001_afff	
fft_yn_imag.s1			
fft_yn_imag.s2		0x0001_9000 - 0x0001_9fff	
fft_yn_re.s1			
fft_yn_re.s2		0x0001_8000 - 0x0001_8fff	
hps_0.f2h_axi_slave			
intr_capturer_0.avalon_slave_0	0x0003_0000 - 0x0003_0007		
jtag_uart.avalon_jtag_slave	0x0002_0000 - 0x0002_0007		0x0002_0000 - 0x0002_0007
onchip_memory2_0.s1	0x0000_0000 - 0x0000_ffff	0x0000_0000 - 0x0000_ffff	
sysid_qsys.control_slave	0x0001_0000 - 0x0001_0007		0x0001_0000 - 0x0001_0007
adc_buf.s1			
adc_buf.s2		0x0002_0000 - 0x0003_ffff	
adc_start.s1			0x0000_00d0 - 0x0000_00df
adc_done.s1			0x0000_00e0 - 0x0000_00ef
adc_config.s1			0x0000_00f0 - 0x0000_00ff
adc_divisor.s1			0x0000_0100 - 0x0000_010f
adc_namostras.s1			0x0000_0110 - 0x0000_011f
adc_contador.s1			0x0000_0120 - 0x0000_012f
soma_teste_a.s1			0x0000_0150 - 0x0000_015f
soma_teste_b.s1			0x0000_0160 - 0x0000_016f
soma_teste_y.s1			0x0000_0170 - 0x0000_017f
soma_a.s1			
soma_a.s2		0x0004_0000 - 0x0004_0fff	
soma_b.s1			
soma_b.s2		0x0004_1000 - 0x0004_1fff (1)	
soma_y.s1			
soma_y.s2		0x0004_2000 - 0x0004_2fff	
soma_start.s1			0x0000_0130 - 0x0000_013f
soma_done.s1			0x0000_0140 - 0x0000_014f (2)

Address Map

B4 O topo e a compilação

O que é e por quê

Igual à A4, com mais fios: cada memória exportada vira 7 portas em `u0` (endereço, dado lido, dado escrito, *chipselect*, *clken*, *write*, *byteenable*), mais as 2 dos PIOs. São 23 fios, cada um ligando uma porta de `u0` a uma porta do `soma.v`.



. . . um fio por sinal: 2 dos PIOs + 7 por memória

Faça

No `ghrd_top.v`, quatro acréscimos, todos **além** dos da A4 (não apague os `soma_teste`: eles continuam servindo para depurar a conta). Use Ctrl+F para achar os lugares.

1. **os parâmetros**: junto dos outros `localparam`, logo depois da linha que tem `localparam ADC_DIV_MIN`:

```
// --- SOMA (modulo de exemplo do roteiro: y[n] = a[n] + b[n], soma.v) ---
localparam SOMA_DATA_WIDTH = 32;
localparam SOMA_ADDR_WIDTH = 10;        // 1024 amostras
localparam SOMA_N_SAMPLES  = 1024;
```

2. **os fios**: logo depois dos três `wire soma_teste` da A4:

```
// --- Controle (PIOs) ---
wire                            soma_start;      // HPS -> FPGA
wire                            soma_done;       // FPGA -> HPS

// --- SRAM a[n]: o HPS escreve, o soma.v le ---
wire [SOMA_ADDR_WIDTH-1:0]      soma_a_address;
wire                            soma_a_clken;
wire                            soma_a_chipselect;
wire                            soma_a_write;
wire [SOMA_DATA_WIDTH-1:0]      soma_a_readdata;
wire [SOMA_DATA_WIDTH-1:0]      soma_a_writedata;
wire [3:0]                      soma_a_byteenable = 4'b1111;

// --- SRAM b[n]: o HPS escreve, o soma.v le ---
wire [SOMA_ADDR_WIDTH-1:0]      soma_b_address;
wire                            soma_b_clken;
wire                            soma_b_chipselect;
wire                            soma_b_write;
wire [SOMA_DATA_WIDTH-1:0]      soma_b_readdata;
wire [SOMA_DATA_WIDTH-1:0]      soma_b_writedata;
wire [3:0]                      soma_b_byteenable = 4'b1111;

// --- SRAM y[n]: o soma.v escreve, o HPS le ---
wire [SOMA_ADDR_WIDTH-1:0]      soma_y_address;
wire                            soma_y_clken;
wire                            soma_y_chipselect;
wire                            soma_y_write;
wire [SOMA_DATA_WIDTH-1:0]      soma_y_readdata;
wire [SOMA_DATA_WIDTH-1:0]      soma_y_writedata;
wire [3:0]                      soma_y_byteenable = 4'b1111;
```

3. **as portas de `u0`**: dentro de `soc_system u0 (`, logo depois da linha `.soma_teste_y_export` da A4. Todas terminam em vírgula, porque há outras portas depois:

```
    .soma_start_export      (soma_start),
    .soma_done_export       (soma_done),

    .soma_a_address         (soma_a_address),
    .soma_a_clken           (soma_a_clken),
    .soma_a_chipselect      (soma_a_chipselect),
    .soma_a_write           (soma_a_write),
    .soma_a_readdata        (soma_a_readdata),
```

```
    .soma_a_writedata       (soma_a_writedata),
    .soma_a_byteenable      (soma_a_byteenable),

    .soma_b_address         (soma_b_address),
    .soma_b_clken           (soma_b_clken),
    .soma_b_chipselect      (soma_b_chipselect),
    .soma_b_write           (soma_b_write),
    .soma_b_readdata        (soma_b_readdata),
    .soma_b_writedata       (soma_b_writedata),
    .soma_b_byteenable      (soma_b_byteenable),

    .soma_y_address         (soma_y_address),
    .soma_y_clken           (soma_y_clken),
    .soma_y_chipselect      (soma_y_chipselect),
    .soma_y_write           (soma_y_write),
    .soma_y_readdata        (soma_y_readdata),
    .soma_y_writedata       (soma_y_writedata),
    .soma_y_byteenable      (soma_y_byteenable),
```

4. **o módulo**: logo depois do `soma_core_teste` da A4, antes do `endmodule`:

```
soma #(
    .DATA_WIDTH  (SOMA_DATA_WIDTH),
    .N_SAMPLES   (SOMA_N_SAMPLES),
    .ADDR_N_BITS (SOMA_ADDR_WIDTH)
) soma_inst (
    .clk     (CLOCK_50),
    .reset_n (hps_fpga_reset_n),

    // PIOs de controle (Platform Designer)
    .start   (soma_start),          // HPS -> FPGA
    .done    (soma_done),           // FPGA -> HPS

    // a[n]: porta s1 da memoria soma_a
    .a_sram_readdata   (soma_a_readdata),
    .a_sram_address    (soma_a_address),
    .a_sram_chipselect (soma_a_chipselect),
    .a_sram_clken      (soma_a_clken),
    .a_sram_write      (soma_a_write),

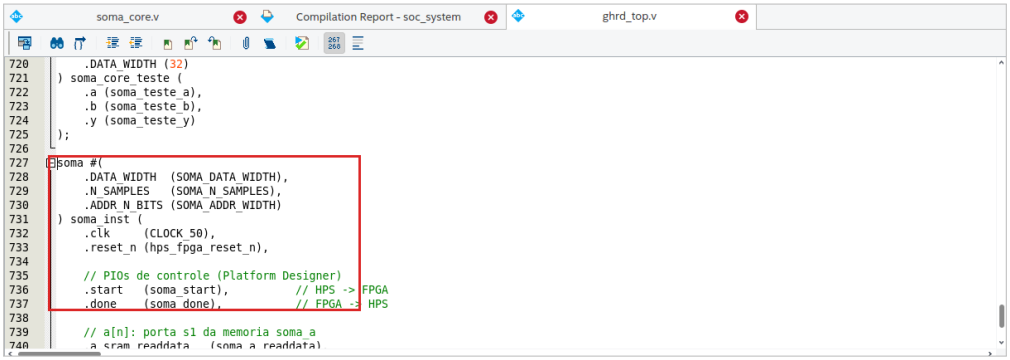
    // b[n]: porta s1 da memoria soma_b
    .b_sram_readdata   (soma_b_readdata),
    .b_sram_address    (soma_b_address),
    .b_sram_chipselect (soma_b_chipselect),
    .b_sram_clken      (soma_b_clken),
    .b_sram_write      (soma_b_write),

    // y[n]: porta s1 da memoria soma_y
    .y_sram_address    (soma_y_address),
    .y_sram_wdata      (soma_y_writedata),
    .y_sram_chipselect (soma_y_chipselect),
    .y_sram_clken      (soma_y_clken),
    .y_sram_write      (soma_y_write)
);
```

Salve (Ctrl+S). Depois: compile, `python3 gen_hps_header.py` e `./morphe-up.sh --cable 'CABO' --board PLACA --deploy`, como na A5 (o mesmo cabo e a mesma placa de lá; se mudou de porta, `jtagconfig` de novo).

O que é e por quê

Cada memória tem duas portas: o HPS usa uma, por dentro de `u0`; a outra é exportada e vem para o `soma.v` por esses fios. Alguns fios vão só até `u0`, e não até o `soma.v`: o `soma.v` só **lê** a e b (o `write` delas fica em 0, e o `writedata` não é usado) e só **escreve** y (o `y_readdata` não é usado). O `byteenable` fixo em `4'b1111` quer dizer "a palavra inteira, os 4 bytes".



a instância `soma_inst` no `ghrd_top.v`

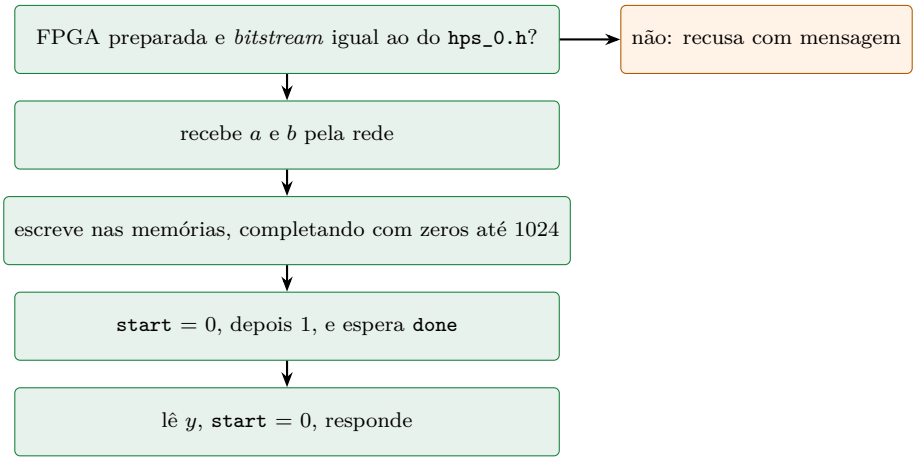
Confira

`grep SOMA_ ../C/hps_0.h` mostra `SOMA_A_BASE 0x40000` e os outros endereços da B1.

B5 O servidor C, na placa

O que é e por quê

O servidor (`C/morphe_server.c`) é um programa que roda no ARM da placa o tempo todo. Ele espera pedidos pela rede; cada pedido traz um **opcode** que diz qual operação, e os dados. O servidor escreve os dados nas memórias da FPGA, dispara o bloco, espera o `done` e devolve o resultado. Para a soma, ele precisa saber: o opcode novo, onde ficam as memórias e os PIOs (do `hps_0.h`) e o que fazer com o pedido.



Faça

Duas formas, escolha uma. A rápida: traga os quatro arquivos prontos do gabarito (de `~/meu-modulo`):

```
git checkout origin/estagio/modulo-soma-adc-fft -- C/morphe_server.c C/morphe_protocol.h
```

```
git checkout origin/estagio/modulo-soma-adc-fft -- C/morphe_config.h C/address_map_arm.h
```

Ou, para entender cada mudança, faça à mão com os trechos completos logo abaixo da tabela. Nos dois

casos, o `C/hps_0.h` **não** vem do gabarito: ele é o que você gerou.

O que mudou em cada um, e por quê:

Arquivo	O que muda	Por quê
`morphe_protocol.h`	`MORPHE_OP_SOMA 9`	o número da operação; o aplicativo usa o mesmo
`morphe_config.h`	`MORPHE_SOMA_N_MAX 1024`	o tamanho máximo aceito, igual ao das memórias
`address_map_arm.h`	`FPGA_ONCHIP_SPAN`: `0x3FFFF` → `0x4FFFF`	o servidor só enxerga a parte da ponte larga que mapeia; as memórias novas ficam depois de `0x40000`
`morphe_server.c`	`MORPHE_TEM_SOMA`	a soma só é compilada se o `hps_0.h` tem os endereços dela
	5 ponteiros em `fpga_init()`	"endereço da ponte + deslocamento do `hps_0.h`" para cada memória e PIO
	`handle_soma()`	o fluxograma acima; o molde foi a `handle_fft()`
	`case MORPHE_OP_SOMA`	o despacho: opcode 9 chama a `handle_soma()`

Acessar a FPGA, em C, é usar um ponteiro — o mesmo que o `devmem` fazia na A6:

```
g_soma_a         = (int32_t *)((char *)g_fpga_virt + SOMA_A_BASE);             /* ponte larga */
g_pio_soma_start = (volatile uint32_t *)((char *)g_lw_virt + SOMA_START_BASE); /* ponte leve  */

g_soma_a[i] = valor;        /* escreve a palavra i da memoria */
*g_pio_soma_start = 1;      /* liga o start */
```

Os trechos completos, arquivo por arquivo

Abra cada arquivo num editor de texto (`gedit`, por exemplo) e use Ctrl+F para achar os lugares.

Faça

`C/morphe_protocol.h`

1. o número da operação: logo depois da linha `#define MORPHE_OP_ADC_CONTINUO`:

```
#define MORPHE_OP_SOMA     9U   /* y = a + b (soma.v), modulo de exemplo do roteiro */
```

2. (só documentação) no comentário do topo, a lista de opcodes passa a ter `8=ADC_CONTINUO, 9=SOMA`, e antes da linha `ADC_CONTINUO -> igual ao ADC` entra:

```
*   SOMA     -> n_x amostras de a, seguidas de n_h amostras de b (int32
*               Q15.16). n_x == n_h, de 1 a MORPHE_SOMA_N_MAX. A
*               resposta traz n_x amostras de y = a + b (int32 Q15.16),
*               saturada em 32 bits.
```

Faça

`C/morphe_config.h`

1. antes da linha `/* ---- Rede`:

```
/* ---- SOMA (modulo de exemplo do roteiro: y = a + b, soma.v) ------------ */

/* O soma.v processa SEMPRE MORPHE_SOMA_N_MAX amostras (N_SAMPLES do
 * Verilog); o servidor completa com zeros e devolve so as n pedidas. As
 * memorias soma_a, soma_b e soma_y tem 1024 palavras de 32 bits cada. */
#define MORPHE_SOMA_N_MAX       1024
```

Faça

`C/address_map_arm.h`

1. **troque** a linha `#define FPGA_ONCHIP_SPAN 0x0003FFFF` por:

```
/* Ate 0x42FFF desde o modulo soma (soma_a/b/y em 0x40000-0x42FFF). Cresce
 * quando uma memoria nova passa do fim: o _Static_assert do servidor avisa. */
#define FPGA_ONCHIP_SPAN       0x0004FFFF
```

Faça

`C/morphe_server.c`, em nove lugares:

1. **no lugar** das duas linhas que começam em `#define MORPHE_ONCHIP_MMAP_SPAN` (logo depois do bloco do ADC, o `#endif` abaixo de `#define _MORPHE_END_ADC 0`); apague as duas e cole:

```
/* A soma, como o ADC, so entra quando o hps_0.h foi gerado de um .sopcinfo
 * que a tem (tres memorias e dois PIOs). A protecao contra bitstream antigo
 * e o sysid, que o servidor ja le por causa do ADC: por isso MORPHE_TEM_ADC
 * tambem e exigido. */
#if defined(SOMA_A_BASE) && defined(SOMA_B_BASE) && defined(SOMA_Y_BASE) && \
    defined(SOMA_START_BASE) && defined(SOMA_DONE_BASE) && MORPHE_TEM_ADC
#define MORPHE_TEM_SOMA 1
#define _MORPHE_END_SOMA \
    _MORPHE_MAX3( \
        _MORPHE_END(SOMA_A_BASE, SOMA_A_SPAN), \
        _MORPHE_END(SOMA_B_BASE, SOMA_B_SPAN), \
        _MORPHE_END(SOMA_Y_BASE, SOMA_Y_SPAN))
#else
#define MORPHE_TEM_SOMA 0
#define _MORPHE_END_SOMA 0
#endif

#define MORPHE_ONCHIP_MMAP_SPAN \
    _MORPHE_ROUND_UP(_MORPHE_MAX3(_MORPHE_END_FFT, _MORPHE_END_ADC, _MORPHE_END_SOMA), \
                     _MORPHE_PAGE_SIZE)
```

2. antes da linha `_Static_assert(MORPHE_ONCHIP_MMAP_SPAN`:

```
#if MORPHE_TEM_SOMA
_Static_assert(SOMA_A_SPAN      >= MORPHE_SOMA_N_MAX * (int)sizeof(int32_t), "Erro");
_Static_assert(SOMA_B_SPAN      >= MORPHE_SOMA_N_MAX * (int)sizeof(int32_t), "Erro");
_Static_assert(SOMA_Y_SPAN      >= MORPHE_SOMA_N_MAX * (int)sizeof(int32_t), "Erro");
#endif
```

3. os ponteiros: depois do `#endif` que vem logo abaixo da linha `static volatile uint32_t *g_pio_adc_contador = NULL;`:

```
#if MORPHE_TEM_SOMA
static int32_t           *g_soma_a          = NULL;
static int32_t           *g_soma_b          = NULL;
static int32_t           *g_soma_y          = NULL;
static volatile uint32_t *g_pio_soma_start  = NULL;
static volatile uint32_t *g_pio_soma_done   = NULL;
#endif
```

4. o nome da operação, nos logs: em `nome_op()`, depois de `case MORPHE_OP_ADC_CONTINUO: return "adc_continuo";`:

```
        case MORPHE_OP_SOMA: return "soma";
```

5. os endereços: em `fpga_init()`, depois do `#endif` que vem logo abaixo da linha `g_pio_adc_contador = ...`:

```
#if MORPHE_TEM_SOMA
    /* memorias: janela da ponte h2f (FPGA_ONCHIP_BASE) + offset do hps_0.h;
     * PIOs: janela da ponte lightweight (LW_BRIDGE_BASE) + offset */
    g_soma_a         = (int32_t *)((char *)g_fpga_virt + SOMA_A_BASE);
    g_soma_b         = (int32_t *)((char *)g_fpga_virt + SOMA_B_BASE);
    g_soma_y         = (int32_t *)((char *)g_fpga_virt + SOMA_Y_BASE);
    g_pio_soma_start = (volatile uint32_t *)((char *)g_lw_virt + SOMA_START_BASE);
```

```
    g_pio_soma_done  = (volatile uint32_t *)((char *)g_lw_virt + SOMA_DONE_BASE);
#endif
```

6. zerar o `start` quando a FPGA fica pronta: em `fpga_preparada()`, depois do `#endif` que vem logo abaixo de `*g_pio_adc_start = 0;`:

```
#if MORPHE_TEM_SOMA
        if (g_sysid[1] == SYSID_QSYS_TIMESTAMP) *g_pio_soma_start = 0;
#endif
```

7. a função que atende o pedido (o fluxograma acima): **antes** da linha `static int handle_ping(int sock) {`:

```
/* SOMA: y = a + b no soma.v. Payload: a (n_x palavras) e depois b (n_h
 * palavras), int32 Q15.16 big-endian. O hardware processa sempre
 * MORPHE_SOMA_N_MAX amostras: o que o cliente nao mandou vai como zero.
 * O molde e a handle_fft(): recebe, escreve nas memorias, dispara, le. */
#if !MORPHE_TEM_SOMA
static int handle_soma(int sock, uint16_t dtype, uint32_t n_a, uint32_t n_b) {
    (void) dtype; (void) n_a; (void) n_b;
    LOG("SOMA request recusado: servidor compilado sem a soma");
    return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_BAD_OPCODE,
                      "SOMA: este servidor foi compilado sem a soma (o hps_0.h nao tem "
                      "soma_a: gere o soc_system e rode gen_hps_header.py)");
}
#else
static int handle_soma(int sock, uint16_t dtype, uint32_t n_a, uint32_t n_b) {
    LOG("SOMA request: dtype=%u, n_a=%u, n_b=%u", dtype, n_a, n_b);

    if (!fpga_preparada()) return recusar_sem_fpga(sock, MORPHE_OP_SOMA, "SOMA");

    /* Sem esta conferencia, um servidor novo numa placa com o bitstream
     * antigo escreveria em 0x40000, onde nao ha memoria -- e acesso a
     * endereco sem escravo trava o barramento do HPS. */
    uint32_t ts = g_sysid[1];
    if (ts != SYSID_QSYS_TIMESTAMP) {
        char msg[192];
        snprintf(msg, sizeof msg,
                 "SOMA: o bitstream na FPGA (sysid %u) nao e o deste servidor (%u) -- "
                 "rode ./morphe-up.sh com o .sof que tem a soma", ts, SYSID_QSYS_TIMESTAMP);
        LOG("  -> %s", msg);
        return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_FPGA_NAO_PREPARADA, msg);
    }

    if (dtype != MORPHE_DTYPE_INT32)
        return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_BAD_DTYPE, "SOMA: use int32 (Q15.16)"
    );
    if (n_a == 0 || n_a > MORPHE_SOMA_N_MAX || n_b != n_a) {
        char msg[96];
        snprintf(msg, sizeof msg, "SOMA: a e b com o mesmo tamanho, de 1 a %d",
                 MORPHE_SOMA_N_MAX);
        return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_BAD_SIZE, msg);
    }
    uint32_t n = n_a;

    /* 1. recebe a e b */
    static uint8_t rx_buf[2 * MORPHE_SOMA_N_MAX * 4];
    if (recv_exact(sock, rx_buf, (size_t) 2 * n * 4) < 0) return -1;

    /* 2. escreve nas memorias da FPGA (porta s2), completando com zeros */
    for (uint32_t i = 0; i < MORPHE_SOMA_N_MAX; i++)
        g_soma_a[i] = (i < n) ? i32_from_be(rx_buf + i * 4) : 0;
    for (uint32_t i = 0; i < MORPHE_SOMA_N_MAX; i++)
        g_soma_b[i] = (i < n) ? i32_from_be(rx_buf + (n + i) * 4) : 0;

    /* 3. dispara (borda de subida no start) e espera o done */
    *g_pio_soma_start = 0;
    usleep(1);
    *g_pio_soma_start = 1;
    if (wait_done(g_pio_soma_done, FPGA_DONE_TIMEOUT_MS) < 0) {
        *g_pio_soma_start = 0;
        return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_FPGA_TIMEOUT, "SOMA: timeout");
    }
```

```
    *g_pio_soma_start = 0;      /* com start em 0, o soma.v volta ao repouso */

    /* 4. le a saida e responde */
    static uint8_t tx_buf[MORPHE_SOMA_N_MAX * 4];
    for (uint32_t i = 0; i < n; i++) i32_to_be(tx_buf + i * 4, g_soma_y[i]);

    uint8_t hdr[MORPHE_HEADER_SIZE];
    build_resp_header(hdr, MORPHE_OP_SOMA, MORPHE_DTYPE_INT32, MORPHE_STATUS_OK, n);
    if (send_all(sock, hdr, sizeof hdr) < 0) return -1;
    if (send_all(sock, tx_buf, (size_t) n * 4) < 0) return -1;

    LOG("  -> SOMA OK: n=%u", n);
    return 0;
}
#endif
```

8. o PING passa a dizer se a soma está disponível: em `handle_ping()`, **troque** tudo da linha `int preparada = fpga_preparada();` até a linha `uptime_s, preparada, fpga_tem_assinatura());` por:

```
    int preparada = fpga_preparada();
    int adc_ok = 0, soma_ok = 0;
#if MORPHE_TEM_ADC
    adc_ok = preparada && g_sysid[1] == SYSID_QSYS_TIMESTAMP;
#endif
#if MORPHE_TEM_SOMA
    soma_ok = adc_ok;          /* o mesmo bitstream, o mesmo sysid */
#endif

    char body[512];
    int body_len = snprintf(body, sizeof body,
        "service=morphe\nversion=%u\nhostname=%s\nfft_n=%d\nfft_data_bits=%d\n"
        "fft_frac_bits=%d\nadc_n_max=%d\nadc_fs_max=%d\n"
        "adc_fs_min=%d\nsoma_n_max=%d\nuptime_s=%ld\nfpga_preparada=%d\nfpga_assinatura=%d\n",
        MORPHE_VERSION, g_hostname, MORPHE_FFT_N, MORPHE_FFT_DATA_BITS,
        MORPHE_FFT_FRAC_BITS,
        adc_ok ? MORPHE_ADC_N_MAX : 0,
        MORPHE_ADC_CLK_HZ / MORPHE_ADC_DIV_MIN, MORPHE_ADC_CLK_HZ / MORPHE_ADC_DIV_MAX,
        soma_ok ? MORPHE_SOMA_N_MAX : 0,
        uptime_s, preparada, fpga_tem_assinatura());
```

9. o despacho: em `serve_connection()`, depois da linha `case MORPHE_OP_ADC_CONTINUO: ...`:

```
        case MORPHE_OP_SOMA: handle_soma(sock, dtype, n_x, n_h); break;
```

Cuidado

A conferência do *bitstream* (*sysid*) no topo do fluxograma não é enfeite: se a placa estiver com um `.sof` sem a soma, escrever em `0x40000` trava a placa, como o `devmem` da A6.

Faça

Mande o servidor novo para a placa. Até aqui você só mudou os arquivos no seu clone; a placa continua rodando o servidor antigo, que **não conhece** o opcode 9. Na estação, de dentro de `~/meu-modulo`, com o mesmo `CABO` e a mesma `PLACA` da A5:

```
./morphe-up.sh --cable 'CABO' --board PLACA --deploy
```

O que é e por quê

O `--deploy` copia os arquivos de `C/` para a pasta `~/morphe` da placa, compila o servidor **lá**, no ARM (`make -B morphe_server`), e o reinicia. Não precisa compilar nada à mão. Toda vez que mudar um arquivo de `C/`, rode-o de novo; se não mudou nada, ele percebe (compara a impressão digital dos arquivos) e só confere a placa.

Confira

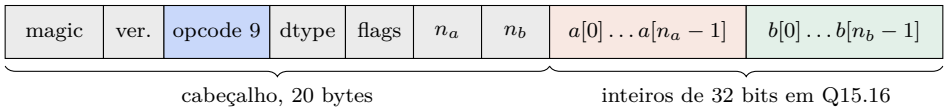
- o `morphe-up.sh` termina com os testes `4/4`;
- na placa (`ssh root@`*IP da placa*), `grep -c MORPHE_OP_SOMA ~/morphe/morphe_server.c` dá um número maior que zero (0 = a placa ainda tem as fontes antigas).

Se, no fim da B6, o aplicativo disser *opcode desconhecido*, é este passo que faltou — ou o aplicativo está ligado na **outra** placa: veja o IP em "Placas do laboratório" e clique em **Usar** na sua.

B6 O aplicativo, e o teste final

O que é e por quê

O aplicativo (`Python/morphe_app.py`) é o que o aluno abre. Para a soma, ele precisa: montar o pedido com o opcode 9 e as amostras, entender a resposta, e uma janela para gerar a e b e mostrar y. O pedido leva os dois sinais, um depois do outro:



Faça

Duas formas, escolha uma. A rápida: traga os quatro arquivos prontos do gabarito:

```
git checkout origin/estagio/modulo-soma-adc-fft -- Python/morphe_protocol.py Python/
    morphe_config.py
```

```
git checkout origin/estagio/modulo-soma-adc-fft -- Python/soma_window.py Python/morphe_app.py
```

Ou faça à mão com os trechos completos logo abaixo do quadro "Já existe". Em Python, a **indentação** (os espaços no começo da linha) faz parte do código: cole cada trecho com os espaços que ele tem.

Arquivo	O que muda	Por quê
`morphe_config.py`	`SOMA_N_MAX = 1024`	o mesmo limite do servidor
`morphe_protocol.py`	`OP_SOMA = 9`, `build_soma_request()`, `decode_soma_response()`	monta os bytes do pedido e confere a resposta
`soma_window.py`	a janela (novo)	dois painéis de sinal, o botão **Somar na FPGA** e os gráficos; confere y com a mesma conta feita no NumPy
`morphe_app.py`	um `import`, uma linha no menu, `_open_soma()`	o botão que abre a janela

Já existe: reaproveite

A janela é curta porque quase tudo já existe: o painel de sinal (`SignalPanel`), o gráfico de amostras da janela da FFT (`_stem_signal`, em `fft_window.py`) e o cliente já apontado para a placa escolhida (`self.master.tcp_panel.make_client()`). Só a conversão para Q15.16 é nova, e cabe em três linhas (`para_q1516`, no começo do `soma_window.py`).

Os trechos completos, arquivo por arquivo

Faça

`Python/morphe_config.py`

1. antes da linha `# ---- Rede:`

```
# ---- SOMA (modulo de exemplo do roteiro: y = a + b, soma.v) ---------------
# O bloco processa SEMPRE SOMA_N_MAX amostras; o servidor completa com zeros.
# Espelho do MORPHE_SOMA_N_MAX do morphe_config.h.
SOMA_N_MAX: int = 1024
```

Faça

`Python/morphe_protocol.py`

1. logo depois da linha `OP_ADC_CONTINUO = 8`:

```
OP_SOMA = 9    # y = a + b (soma.v), modulo de exemplo do roteiro
```

2. o pedido e a resposta: antes da linha `class TcpClient:`:

```
def build_soma_request(a_q: np.ndarray, b_q: np.ndarray) -> bytes:
    """Pedido da SOMA: cabecalho com n_x = len(a) e n_h = len(b), depois as
    amostras de a e as de b. a e b ja em Q15.16 (inteiros) e com o mesmo
    tamanho, de 1 a SOMA_N_MAX."""
    if len(a_q) != len(b_q):
        raise ValueError(f"a e b precisam do mesmo tamanho ({len(a_q)} != {len(b_q)})")
    hdr = struct.pack(
        ">IHHHHII",
        MAGIC_REQ, VERSION, OP_SOMA, DTYPE_INT32, 0,
        len(a_q), len(b_q),
    )
    return hdr + pack_samples(a_q, DTYPE_INT32) + pack_samples(b_q, DTYPE_INT32)


def decode_soma_response(resp: Response) -> np.ndarray:
    """y = a + b em Q15.16 (inteiros), como a placa devolveu."""
    if not resp.ok:
        raise RuntimeError(f"erro do servidor: {resp.payload.decode('utf-8', 'replace')}")
    if resp.opcode != OP_SOMA:
        raise ValueError(f"esperado opcode SOMA, veio {resp.opcode}")
    be = _np_dtype_be(resp.dtype_code)
    return np.frombuffer(resp.payload, dtype=be, count=resp.n_out).astype(np.int64)
```

Faça

`Python/morphe_app.py`

1. logo depois da linha `from comparator_window import ComparatorWindow`:

```
from soma_window import SomaWindow
```

2. o botão do menu: em `primary_actions`, depois da linha do `"FFT (1 sinal → FPGA)"`:

```
            ("Soma  (a + b → FPGA, exemplo do roteiro)", self._open_soma),
```

3. o que o botão faz: logo depois do método `_open_fft` (as duas linhas dele):

```
    def _open_soma(self):
        SomaWindow(self)
```

Faça

`Python/soma_window.py` (arquivo novo): crie e cole o arquivo inteiro.

```
"""
```

```
soma_window.py -- janela da SOMA: y[n] = a[n] + b[n] calculada na FPGA.

Modulo de exemplo do roteiro "Como criar um modulo no Quartus e liga-lo ao
Morphe" (docs/roteiro-modulo). Mostra o minimo que uma janela nova precisa,
reaproveitando o que ja existe:

  - os dois sinais vem do SignalPanel, o mesmo de todas as janelas;
  - o desenho e o _stem_signal da janela da FFT;
  - a placa e a do painel "Placas do laboratorio" (master.tcp_panel);
  - o pedido vai em uma thread, para a janela nao congelar esperando a rede.

A referencia e a mesma conta em ponto fixo no NumPy, com a mesma saturacao
do soma.v: a comparacao com a FPGA tem de dar igualdade BIT A BIT.
"""
from __future__ import annotations

import threading
import tkinter as tk
from tkinter import ttk, messagebox
from typing import Optional

import numpy as np

import matplotlib
matplotlib.use("TkAgg")
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

import dsp_core as dsp
import morphe_theme as theme
from fft_window import _stem_signal
from morphe_config import SOMA_N_MAX
from morphe_protocol import build_soma_request, decode_soma_response
from plot_toolbar import PlotToolbar
from signal_panel import SignalPanel

# Q15.16: o inteiro de 32 bits guardado e o valor vezes 2^16.
Q1516_ESCALA = 1 << 16
Q1516_MIN = -(1 << 31)
Q1516_MAX = (1 << 31) - 1


def para_q1516(x: np.ndarray) -> np.ndarray:
    """Valores reais -> inteiros Q15.16 (arredonda e satura em 32 bits)."""
    q = np.round(np.asarray(x, dtype=np.float64) * Q1516_ESCALA)
    return np.clip(q, Q1516_MIN, Q1516_MAX).astype(np.int64)


def de_q1516(q: np.ndarray) -> np.ndarray:
    """Inteiros Q15.16 -> valores reais."""
    return np.asarray(q, dtype=np.float64) / Q1516_ESCALA


def soma_referencia(a_q: np.ndarray, b_q: np.ndarray) -> np.ndarray:
    """A conta do soma.v em NumPy: soma em 64 bits e satura na faixa de 32."""
    s = np.asarray(a_q, dtype=np.int64) + np.asarray(b_q, dtype=np.int64)
    return np.clip(s, Q1516_MIN, Q1516_MAX)


class SomaWindow(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)
        self.title("Morphe -- Soma via FPGA (exemplo do roteiro)")
        self.geometry("1240x820")
        self.configure(bg=theme.COLORS["bg"])
        theme.setup_styles(self)

        self.a_sig: Optional[dsp.Signal] = None
        self.b_sig: Optional[dsp.Signal] = None
        self.y_sig: Optional[dsp.Signal] = None

        self.status = tk.StringVar(value="Gere a[n] e b[n] e clique em Somar na FPGA.")
        theme.make_status_bar(self, self.status).pack(side="bottom", fill="x")

        root = ttk.Frame(self, style="Main.TFrame", padding=12)
```

```
        root.pack(fill="both", expand=True)
        lado = ttk.Frame(root, style="Main.TFrame")
        lado.pack(side="left", fill="y", padx=(0, 12))
        self._build_sidebar(lado)
        area = ttk.Frame(root, style="Main.TFrame")
        area.pack(side="right", fill="both", expand=True)
        self._build_plots(area)
        self._redraw()

    # ------------------------------------------------------------------
    def _build_sidebar(self, parent):
        self.a_panel = SignalPanel(parent, title="Sinal a[n]", on_generate=self._on_gen_a,
                                   default_N=32, default_type="Senóide", max_N=SOMA_N_MAX)
        self.a_panel.pack(fill="x")
        self.b_panel = SignalPanel(parent, title="Sinal b[n]", on_generate=self._on_gen_b,
                                   default_N=32, default_type="Degrau unitário", max_N=SOMA_N_MAX)
        self.b_panel.pack(fill="x", pady=(8, 0))

        op = ttk.LabelFrame(parent, text=" Operação ", style="Card.TLabelframe",
                            padding=(14, 12, 14, 14))
        op.pack(fill="x", pady=(12, 0))
        ttk.Label(op, style="SectionHint.TLabel", wraplength=300, justify="left",
                  text=f"Até {SOMA_N_MAX} amostras, em Q15.16. Se os tamanhos forem "
                       "diferentes, o menor é completado com zeros.").pack(anchor="w", pady=(0, 8))
        self.btn = ttk.Button(op, text="Somar na FPGA", style="Primary.TButton",
                              command=self._on_somar)
        self.btn.pack(fill="x")

    def _build_plots(self, parent):
        self.fig = Figure(figsize=(7, 8), dpi=100)
        self.fig.patch.set_facecolor(theme.COLORS["bg"])
        self.ax_a = self.fig.add_subplot(311)
        self.ax_b = self.fig.add_subplot(312)
        self.ax_y = self.fig.add_subplot(313)
        self.canvas = FigureCanvasTkAgg(self.fig, master=parent)
        self.canvas.get_tk_widget().pack(fill="both", expand=True)
        self.toolbar = PlotToolbar(self.canvas)

    def _redraw(self):
        _stem_signal(self.ax_a, self.a_sig, "a[n]", dsp.COLOR_X)
        _stem_signal(self.ax_b, self.b_sig, "b[n]", dsp.COLOR_X)
        _stem_signal(self.ax_y, self.y_sig, "y[n] = a[n] + b[n] (FPGA)", dsp.COLOR_Y)
        self.fig.tight_layout()
        self.canvas.draw_idle()

    # ------------------------------------------------------------------
    def _on_gen_a(self, panel: SignalPanel):
        try:
            self.a_sig = panel.build()
            self._redraw()
        except Exception as e:
            messagebox.showerror("Erro em a[n]", str(e))

    def _on_gen_b(self, panel: SignalPanel):
        try:
            self.b_sig = panel.build()
            self._redraw()
        except Exception as e:
            messagebox.showerror("Erro em b[n]", str(e))

    def _on_somar(self):
        if self.a_sig is None or self.b_sig is None:
            messagebox.showwarning("Atenção", "Gere a[n] e b[n] antes.")
            return
        try:
            client = self.master.tcp_panel.make_client()
        except Exception as e:
            messagebox.showerror("Configuração", str(e))
            return

        # mesmo tamanho para os dois; o indice n vem de a[n]
        n = max(self.a_sig.x.size, self.b_sig.x.size)
        a_q = para_q1516(dsp.pad_zeros_to(self.a_sig.x, n))
        b_q = para_q1516(dsp.pad_zeros_to(self.b_sig.x, n))
        n0 = int(self.a_sig.n[0]) if self.a_sig.n.size else 0
```

```
        self.btn.config(state="disabled")
        self.status.set(f"Enviando {n} amostras de a e de b à FPGA...")

        def worker():
            try:
                resp = client.request(build_soma_request(a_q, b_q))
                y_q = decode_soma_response(resp)
                self.after(0, lambda: self._on_resultado(y_q, a_q, b_q, n0))
            except Exception as e:
                self.after(0, lambda err=e: self._on_erro(err))

        threading.Thread(target=worker, daemon=True).start()

    def _on_resultado(self, y_q, a_q, b_q, n0):
        ref = soma_referencia(a_q, b_q)
        iguais = int(np.sum(y_q == ref))
        n = y_q.size
        self.y_sig = dsp.Signal(n=np.arange(n) + n0, x=de_q1516(y_q),
                                description=f"{n} amostras, Q15.16")
        self._redraw()
        self.btn.config(state="normal")
        if iguais == n:
            self.status.set(f"OK: as {n} amostras da FPGA são iguais, bit a bit, "
                            "à soma em ponto fixo feita no NumPy.")
        else:
            self.status.set(f"ATENÇÃO: só {iguais} de {n} amostras iguais à referência.")

    def _on_erro(self, err: Exception):
        self.btn.config(state="normal")
        self.status.set("Erro -- ver mensagem.")
        messagebox.showerror("Erro na soma", str(err))
```

Faça

O teste final. De `~/meu-modulo`, num computador do laboratório:

```
python3 Python/morphe_app.py
```

Em **Placas do laboratório**, confira que a sua placa está marcada *conectado* (se não, clique em **Usar** na linha dela). Depois:

1. no menu, ❶ **Soma (a + b → FPGA, exemplo do roteiro)**;
2. na janela que abre, ❶ **Gerar** em $a[n]$ e ❷ **Gerar** em $b[n]$ (os tipos e tamanhos podem ficar como estão);
3. ❸ **Somar na FPGA**, e leia a barra de baixo ❹.



o botão novo no menu



o que você deve ver: a barra de baixo diz *iguais, bit a bit* (placa 2, 29/10/2026)

Confira

- a barra de baixo diz que as amostras são **iguais, bit a bit** à conta feita no NumPy;
- $a = b = 30\,000$ dá 32 767,99998 (saturou), como na A6;
- as outras janelas (Aquisição, FFT, Comparador) continuam funcionando;
- numa placa com um `.sof` antigo, a soma é **recusada** com mensagem — e a placa não trava.

Faça

Devolver a placa à turma, sempre, ao terminar. Na estação, de `~/meu-modulo`:

```
./morphe-up.sh --down
```

e de `/opt/morphe`:

```
cd /opt/morphe
```

```
./morphe-up.sh --cable 'CABO' --board PLACA
```

com o mesmo `CABO` e a mesma `PLACA` da A5.

Para o seu próprio módulo

Se o seu módulo...	**O que muda em relação à soma**
faz outra conta amostra a amostra, com 1 ou 2 entradas	só o núcleo (`soma_core.v`); copie o resto trocando o nome
tem parâmetros (um ganho, coeficientes)	mais um PIO (poucos valores) ou mais uma memória (muitos)
precisa de mais de 1024 amostras	o tamanho das memórias, o `N_SAMPLES`, os `_N_MAX` e os endereços (veja os buracos livres no **Address Map**)
tem latência de vários ciclos	o `soma.v` espera um sinal de "pronto" do núcleo antes de gravar

Erros comuns

Sintoma	**Causa e solução**
faixa amarela *IP upgrade recommended*, ou *Used ... 23.1 (instead of 20.1)*	abriu com outro Quartus; feche sem salvar e abra o 20.1
Errors na barra de baixo do Platform Designer	porta sem relógio ou reset, ou dois componentes no mesmo endereço (veja o **Address Map**)
can't find port ... `_export` ao compilar	o nome no `ghrd_top.v` não é o gerado; confira no `soc_system/synthesis/soc_system.v`
o Quartus não acha o módulo	`.v` fora de `Quartus/` ou fora da lista do projeto
`devmem` lê sempre 0	o `external_connection` não foi exportado, ou o fio não chega ao módulo no `ghrd_top.v`
a placa some da rede	acesso a endereço sem componente: `devmem` antes de gravar, ou `hps_0.h` diferente do `.sof`
o aplicativo diz *opcode desconhecido*	o servidor da placa é o antigo: faltou o `--deploy` do fim da B5, ou o aplicativo está ligado na outra placa (veja "Placas do laboratório")
operações antigas param de funcionar	usou *Assign Base Addresses*: os endereços mudaram
`_Static_assert` falha ao compilar o servidor	faltou o `FPGA_ONCHIP_SPAN`
a placa para depois de cerca de uma hora	sem a licença ativa; use o `morphe-up.sh`, sem `sudo`
`qsys-generate` ou `quartus_sh`: *not found*	terminal em `ksh`; digite `bash`
`morphe-up.sh`: *não sei qual placa é a deste comando*	faltou o `--board PLACA`, com o nome que aparece no `login:` da serial (A5)
`morphe-up.sh`: *placa em uso*	alguém mandou uma operação a ela no último minuto (pode ter sido você mesmo); espere um minuto e rode de novo